Lecture Notes for Category Theory

Niels van der Weide (vanderweide@inf.elte.hu)

Contents

Preface **3**

1 Categories **4**
1.1 Categories and Examples . . . 4
1.2 Special Morphisms . . . 14
1.3 Exercises . . . 17

2 Constructions in Categories **19**
2.1 Terminal and Initial Objects . . . 19
2.2 Binary Products and Coproducts . . . 21
2.3 Pullbacks . . . 26
2.4 Exercises . . . 28

3 Functors and Natural Transformations **30**
3.1 Functors . . . 30
3.2 Natural Transformations . . . 35
3.3 Categories of Functors . . . 38
3.4 Exercises . . . 41

4 Algebras and Coalgebras **42**
4.1 Algebras for a Functor . . . 42
4.2 Coalgebras for a Functor . . . 48
4.3 Exercises . . . 50

5 Adjunctions **51**

6 Monads **52**

7 Presheaves and the Yoneda Lemma **53**

8 Cartesian Closed Categories **54**

9 Cartesian Closed Categories and the λ-Calculus **55**

10 Hyperdoctrines **56**

11 Quantifiers as Adjoints **57**

12 Triposes and Higher-Order Logic **58**

Preface

preface

1 Categories

1.1 Categories and Examples

Definition 1.1. A **category** $\mathcal{C}$ consists of

- a class $\mathcal{C}_0$ of **objects**;
- for all objects $x, y \in \mathcal{C}_0$, a class $\mathcal{C}_1(x, y)$ of **morphisms**;
- for all objects $x \in \mathcal{C}_0$, a morphism $\mathsf{id}_x : \mathcal{C}_1(x, x)$ called the **identity morphism**;
- for all objects $x, y, z \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(x, y)$ and $g : \mathcal{C}_1(y, z)$, a morphism $g \circ_{\mathcal{C}} f : \mathcal{C}_1(x, z)$ called the **composition**.

We require the following equations

- for all objects $x, y \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(x, y)$, we have $\mathsf{id}_y \circ_{\mathcal{C}} f = f$;
- for all objects $x, y \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(x, y)$, we have $f \circ_{\mathcal{C}} \mathsf{id}_x = f$;
- for all objects $w, x, y, z \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(w, x)$, $g : \mathcal{C}_1(x, y)$, and $h : \mathcal{C}_1(y, z)$, we have $h \circ_{\mathcal{C}} (g \circ_{\mathcal{C}} f) = (h \circ_{\mathcal{C}} g) \circ_{\mathcal{C}} f$.

The equations $\mathsf{id}_y \circ_{\mathcal{C}} f = f$ and $f \circ_{\mathcal{C}} \mathsf{id}_x = f$ say that the morphism id is a unit element for composition, and we call them the **left and right unitality laws**. The equation $f \circ_{\mathcal{C}} (g \circ_{\mathcal{C}} h) = (f \circ g) \circ_{\mathcal{C}} h$ says that composition is **associative**, and this equation makes $f \circ_{\mathcal{C}} g \circ_{\mathcal{C}} h$ unambiguous.

To lighten the notation, we often write $x \in \mathcal{C}$ instead of $x \in \mathcal{C}_0$. We also write $f : x \to_{\mathcal{C}} y$ instead of $f \in \mathcal{C}_1(x, y)$, and if $\mathcal{C}$ is clear from the context, we write $f : x \to y$ instead. In addition, we often write id instead of id_x if x is clear from the context, and we write $f \cdot g$ for $f \cdot_{\mathcal{C}} g$ if $\mathcal{C}$ is clear from the context.

Digression. In Definition 1.1 we made a distinction between sets and classes. Such a distinction is necessary to define, for instance, the category of sets (Example 1.2). The objects of this category should be all sets, but as there is no sets of all sets, such a category cannot exist if we require the objects of a category to form a set. We solve this problem by relaxing the definition of category to allow the objects and morphisms to form a class rather than a set. Another solution would be restrict this category to all set whose cardinality is at most κ for some cardinal number κ. However, there is a disadvantage to this solution: we need κ to be an inaccessible cardinal in order to show that the resulting category of sets has desirable properties (e.g., Cartesian closed). Throughout these notes, we disregard issues regarding sets versus classes.

Let us discuss some examples of categories. We start by examples of categories that arise from mathematical structures. In Table 1, we summarise the examples of categories that we discuss throughout this section.

Example 1.2. We define the **category of sets**, written as Set, as follows

- the objects of Set are sets;
- the morphisms from a set X to a set Y are functions from X to Y.

The identity morphism $\mathsf{id} : X \to X$ is defined to be the identity function, i.e., $\mathsf{id}(x) = x$. The composition of morphisms $f : X \to Y$ and $g : Y \to Z$ is defined to be the composition of functions, i.e., $(g \circ f)(x) = g(f(x))$. Let $f : X \to Y$ be a morphism. Left unitality ($\mathsf{id} \circ f = f$) follows from the following equations, where $x \in X$ is an arbitrary element.

$$\begin{aligned}(\mathsf{id} \circ f)(x) &= \mathsf{id}(f(x)) \\ &= f(x)\end{aligned}$$

Right unitality ($f \circ \mathsf{id} = f$) is proven similarly.

$$\begin{aligned}(f \circ \mathsf{id})(x) &= f(\mathsf{id}(x)) \\ &= f(x)\end{aligned}$$

Now suppose we have morphisms $f : W \to X$, $g : X \to Y$, and $h : Y \to Z$. To prove associativity, we take an arbitrary $w \in W$, and we use the following equalities.

$$\begin{aligned}(h \circ (g \circ f))(w) &= h((g \circ f)(w)) \\ &= h(g(f(w))) \\ &= (h \circ g)(f(w)) \\ &= ((h \circ g) \circ f)(w)\end{aligned}$$

Hence, Set indeed forms a category.

Example 1.3. A **monoid** consists of a set M together an element $e_M \in M$ and an operation $m : M \times M \to M$ such that the following equations holds

- $m(e_M, x) = x$ for all $x \in M$;
- $m(x, e_M) = x$ for all $x \in M$;
- $m(x, m(y, z)) = m(m(x, y), z)$ for all $x, y, z \in M$.

We denote $m(x, y)$ by $x \cdot y$, and we denote a monoid by M. A **homomorphism** from a monoid M to a monoid N is a function $f : M \to N$ such that $f(e_M) = e_N$ and $f(x \cdot y) = f(x) \cdot f(y)$. The identity function $\mathsf{id} : M \to M$ is a homomorphism, because

$$\mathsf{id}(e_M) = e_M, \qquad \mathsf{id}(x \cdot y) = x \cdot y = \mathsf{id}(x) \cdot \mathsf{id}(y)$$

If $f : M \to N$ and $g : N \to K$ are homomorphisms, then their composition is a homomorphism as well since

$$(g \circ f)(e_M) = g(f(e_M)) = g(e_N) = e_K$$

$$(g \circ f)(x \cdot y) = g(f(x \cdot y)) = g(f(x) \cdot f(y)) = g(f(x)) \cdot g(f(y))$$

Note that two homorphisms $f, g : M \to N$ are the same if their underlying functions are the same, meaning that to prove $f = g$, it suffices to show that $f(m) = g(m)$ for all $m \in M$.

Now we define the **category of monoids**, writen Mon, as follows.

- the objects of Mon are monoids;
- the morphisms from a monoid M to a monoid N are homomorphisms from M to N.

The identity $\mathsf{id} : M \to M$ is given by the identity homomorphism, and the composition is given by the composition of homomorphisms. We prove that Mon is a category in the same way as we prove that Set is a category. To prove left unitality ($\mathsf{id} \circ f = f$ for all homomorphisms f), we take an arbitrary $m \in M$, and we note

$$\begin{aligned}(\mathsf{id} \circ f)(m) &= \mathsf{id}(f(m)) \\ &= f(m)\end{aligned}$$

Example 1.4. A **group** G consists of a set G together an element $e_G \in G$ and operations $i : G \to G$ and $m : G \times G \to G$ such that the following equations holds

- $m(e_G, g) = g$ for all $g \in G$;
- $m(g, e_G) = g$ for all $g \in G$;
- $m(g, m(h, k)) = m(m(g, h), k)$ for all $g, h, k \in G$;
- $m(g, i(g)) = e_G$ for all $g \in G$;
- $m(i(g), g) = e_G$ for all $g \in G$.

We denote $m(g, h)$ by $g \cdot h$, and we denote $i(g)$ by g^{-1}. A **homomorphism** from a group G to a group H is a function $f : G \to H$ such that $f(e_G) = e_H$ and $f(g \cdot h) = f(g) \cdot f(h)$. Note that we can prove $f(g^{-1}) = (f(g))^{-1}$.

We define the **category of groups**, written Group, as follows

- the objects of Group are groups;
- the morphisms from a group G to a monoid H are homomorphisms from G to H.

We prove that Group is a category the same way as we proved that Mon is a category.

Another example of a category is the category PreOrd (Exercise 1.4). Next we look at constructions of categories.

Example 1.5. We write $\mathbb{1}$ for the set $\{*\}$. We define the **unit category**, denoted by unit, as follows.

- The objects of unit are elements of $\mathbb{1}$;
- The morphisms from $*$ to $*$ are elements of $\mathbb{1}$

The identity morphism is given by $*$, and the composition of morphisms is the constant maps that maps everything to $*$. The necessary laws follow directly, because all elements of $\mathbb{1}$ are equal, and hence all morphisms in unit are equal.

Example 1.6. Let $\mathcal{C}$ be a category. We define **opposite category**, denoted by $\mathcal{C}^{\mathsf{op}}$, as follows.

- The objects of $\mathcal{C}^{\mathsf{op}}$ are objects of $\mathcal{C}$;
- The morphisms from x to y in $\mathcal{C}^{\mathsf{op}}$ are morphisms from y to x in $\mathcal{C}$.

Hence, $\mathcal{C}^{\mathsf{op}}$ is $\mathcal{C}$ but with the morphisms reversed. The identity morphism $\mathsf{id}_x : \mathcal{C}_1^{\mathsf{op}}(x,x)$ in $\mathcal{C}^{\mathsf{op}}$ is the identity morphism $\mathsf{id}_x : \mathcal{C}_1(x,x)$ in $\mathcal{C}$. Suppose that we have morphisms $f : \mathcal{C}_1^{\mathsf{op}}(x,y)$ and $g : \mathcal{C}_1^{\mathsf{op}}(y,z)$, which are the same as morphisms $f : \mathcal{C}_1(y,x)$ and $g : \mathcal{C}_1(z,y)$. We define the composition $g \circ_{\mathcal{C}^{\mathsf{op}}} f$ to be $f \circ_{\mathcal{C}} g$. Left unitality is proven as follows

$$\begin{aligned} \mathsf{id} \circ_{\mathcal{C}^{\mathsf{op}}} f &= f \circ_{\mathcal{C}} \mathsf{id} \\ &= f \end{aligned}$$

Here we use right unitality of $\mathcal{C}$. Right unitality for $\mathcal{C}^{\mathsf{op}}$ is left as an exercise. To prove associativity for $\mathcal{C}^{\mathsf{op}}$, we assume that we have morphisms $f : \mathcal{C}_1^{\mathsf{op}}(w,x)$, $g : \mathcal{C}_1^{\mathsf{op}}(x,y)$, and $h : \mathcal{C}_1^{\mathsf{op}}(y,z)$. Note that we have the following equalities.

$$\begin{aligned} h \circ_{\mathcal{C}^{\mathsf{op}}} (g \circ_{\mathcal{C}^{\mathsf{op}}} f) &= (g \circ_{\mathcal{C}^{\mathsf{op}}} f) \circ_{\mathcal{C}} h \\ &= (f \circ_{\mathcal{C}} g) \circ_{\mathcal{C}} h \\ &= f \circ_{\mathcal{C}} (g \circ_{\mathcal{C}} h) \\ &= (g \circ_{\mathcal{C}} h) \circ_{\mathcal{C}^{\mathsf{op}}} f \\ &= (h \circ_{\mathcal{C}^{\mathsf{op}}} g) \circ_{\mathcal{C}^{\mathsf{op}}} f \end{aligned}$$

The compositions above make sense, because $f : \mathcal{C}_1^{\mathsf{op}}(w,x)$, $g : \mathcal{C}_1^{\mathsf{op}}(x,y)$, and $h : \mathcal{C}_1^{\mathsf{op}}(y,z)$ are the same as morphisms $f : \mathcal{C}_1(x,w)$, $g : \mathcal{C}_1(y,x)$, and $h : \mathcal{C}_1(z,y)$ in $\mathcal{C}$.

Example 1.7. Suppose that we have categories $\mathcal{C}_1$ and $\mathcal{C}_2$. We define the **product category**, denoted by $\mathcal{C}_1 \times \mathcal{C}_2$, as follows.

- The objects of $\mathcal{C}_1 \times \mathcal{C}_2$ are pairs (x,y) where $x \in \mathcal{C}_1$ and $y \in \mathcal{C}_2$;
- The morphisms from (x_1,y_1) to (x_2,y_2) in $\mathcal{C}_1 \times \mathcal{C}_2$ are pairs (f,g) of morphisms where $f : x_1 \to_{\mathcal{C}_1} y_1$ and $g : x_2 \to_{\mathcal{C}_2} y_2$.

We leave it as an exercise to construct the identity morphisms and the composition of morphisms, and to prove that it indeed forms a category.

Example 1.8. Let $\mathcal{C}$ be a category and let P be a predicate on the objects of $\mathcal{C}$. We define the **full subcategory** $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ as follows.

- The objects of $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ are objects of $x \in \mathcal{C}$ such that $\mathsf{P}(x)$
- Given objects $x, y \in C$ such that $\mathsf{P}(x)$ and $\mathsf{P}(y)$, then morphisms from x to y in $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ are the same as morphisms $x \to_{\mathcal{C}} y$.

The identity morphisms and composition operations in $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ are inherited from $\mathcal{C}$, and we leave it as an exercise to the reader to verify that we indeed have a category.

Next we show that categories generalize certain mathematical structures. Specifically, we show how to construct categories from other mathematical structures.

Example 1.9. A **preorder** consists of a set X together with a relation $\leq$ on X that is reflexive ($x \leq x$ for each x) and transitive (if $x \leq y$ and $y \leq z$, then $x \leq z$). Every preorder $(X, \leq)$ gives rise to a category $\widetilde{(X, \leq)}$ as follows.

- The objects of $\widetilde{(X, \leq)}$ are elements of X;
- There is a unique morphism from $x \in X$ to $y \in X$ if and only if $x \leq y$.

Since $\leq$ is reflexive, we have an identity morphism. To construct the composition of morphisms in $\widetilde{(X, \leq)}$, we use that $\leq$ is transitive. The desired laws follow from the fact that morphisms in $\widetilde{(X, \leq)}$ are unique.

Example 1.10. Let M be a monoid. We define the category $\mathsf{B}(M)$ as follows.

- The objects of $\mathsf{B}(M)$ is $\mathbb{1}$;
- morphisms from $*$ to $*$ are the same as elements of M

The identity morphism is given by e_M, and if we have morphisms $m_1, m_2 : * \to$ *unitt*, then their composition $m_1 \circ_{\mathsf{B}(M)} m_2$ is given by $m_1 \cdot m_2$. The laws are proven as follows.

$$m \circ_{\mathsf{B}(M)} \mathsf{id} = m \cdot e_M = m$$

$$\mathsf{id} \circ_{\mathsf{B}(M)} m = e_M \cdot m = m$$

$$m_1 \circ_{\mathsf{B}(M)} (m_2 \circ_{\mathsf{B}(M)} m_3) = m_1 \cdot (m_2 \cdot m_3) = (m_1 \cdot m_2) \cdot m_3 = (m_1 \circ_{\mathsf{B}(M)} m_2) \circ_{\mathsf{B}(M)} m_3$$

Since every group is in particular a monoid, we can instantiate Example 1.10 to groups as well.

Example 1.11 (Extra)**.** For the final example, we assume familiarity with the **λ-calculus**. Recall that the terms of the λ-calculus are defined using the following grammar

$$\Lambda := x \mid \Lambda\Lambda \mid \lambda x.\Lambda$$

Arbitrary terms are denoted by M. Types in the λ-calculus are defined using the following grammar.

$$\mathsf{Ty} := \iota \mid \mathsf{Ty} \Rightarrow \mathsf{Ty}$$

Here ι is the base type, and we denote arbitrary types by τ and σ. The typing relation $x_1 : \tau_1, \ldots, x_n : \tau_n \vdash M : \sigma$ is defined using the usual derivation rules We define the **syntactic category of the simply typed λ-calculus**, denoted by STLC, as follows.

- The objects of STLC are types
- The morphisms from τ to σ are equivalence classes of terms t such that $x : \tau \vdash t : \sigma$, where terms are identified using $\beta\eta$-conversion.

The identity morphism from τ to τ is given by the variable $x : \tau \vdash x : \tau$, and if we have terms $x : \tau_1 \vdash s : \tau_2$ and $y : \tau_2 \vdash t : \tau_3$, then their composition is $x : \tau_1 \vdash t[y \mapsto s] : \tau_3$.

Digression. Example 1.1 is inspiring: we can turn various typed languages into categories. For instance, we can extend to include various types, such as sum-types and product types. One might wonder: can we do the same for Haskell? Can we make a category whose objects are types (as in Haskell) and whose morphisms are Haskell functions. Doing so comes with various complications. Since Haskell's type system includes polymorphism, we need to take into account that Haskell types could have type variables. However, there is an even more fundamental issue. Haskell has a function $\mathsf{seq} : a \to b \to b$ that evaluates its first argument before returning its second, and we also have a function $\mathsf{undefined} : a \to b$ that is undefined for each argument. Now we can show the following

$$\mathsf{seq}\ \mathsf{undefined}\ () = \mathsf{undefined}$$

$$\mathsf{seq}\ (\mathsf{undefined} \circ \mathsf{id})() = ()$$

Note that $\mathsf{undefined} \circ \mathsf{id}$ evaluates to itself, which is why the second equation holds. However, from these equations we can conclude that $\mathsf{undefined}$ and $\mathsf{undefined} \circ \mathsf{id}$ cannot be equal, and thus the category laws do not hold.

When we talk about categories, we generally use the language of **diagrams**. A diagram in a category $\mathcal{C}$ is given by a graph whose nodes are objects in $\mathcal{C}$ and whose edges from x to y are labelled by a morphism $f : x \to_{\mathcal{C}} y$. For instance, suppose that we have a category $\mathcal{C}$ and let v, w, x, y, z be objects in $\mathcal{C}$. Suppose that we have morphisms

$$f_1 : v \to w, \quad f_2 : w \to x, \quad f_3 : x \to z, \quad f_4 : w \to y, \quad f_5 : v \to y, \quad f_6 : y \to z$$

An example of a diagram is given below

$$\begin{tikzcd}
& w \arrow[r, "f_2"] \arrow[dd, "f_4"'] & x \arrow[dd, "f_3"] \\
v \arrow[ur, "f_1"] \arrow[dr, "f_5"'] & & \\
& y \arrow[r, "f_6"'] & z
\end{tikzcd}$$

Each path in this graph gives rise to a composition of morphisms. For instance, we have two paths from v to y, namely $f_4 \circ f_1$ and f_1. We have three paths from v to z, which are given by the following compositions

$$f_3 \circ f_2 \circ f_1, \quad f_6 \circ f_4 \circ f_1, \quad f_6 \circ f_5$$

Since composition is associative, every path describes a unique morphism. A diagram is said to be **commutative** if for all nodes x and y every path from x to y denotes the same morphism.

To get familiar with the language of diagrams, we look at other examples of categories.

Example 1.12. Let $\mathcal{C}$ be a category and let x be an object in $\mathcal{C}$. We define the **slice category** $\mathcal{C}/x$ as follows.

- The objects of $\mathcal{C}/x$ are pairs of an object $y \in Y$ together with a morphism $f : y \to x$.
- Given objects $y_1, y_2 \in \mathcal{C}$ and morphisms $f_1 : y_1 \to x$ and $f_2 : y_2 \to x$, then morphisms from (y_1, f_1) to (y_2, f_2) in $\mathcal{C}/x$ are morphisms $g : y_1 \to y_2$ such that the following diagram commutes.

$$\begin{tikzcd} y_1 \arrow[rr, "g"] \arrow[dr, "f_1"'] & & y_2 \arrow[dl, "f_2"] \\ & x & \end{tikzcd}$$

  Concretely, this means that $f_1 = f_2 \circ g$.

The identity morphism on (y, f) is given by id_y. Note that id_y indeed gives a morphism in $\mathcal{C}/x$ since the following diagram commutes

$$\begin{tikzcd} y \arrow[rr, "\mathsf{id}_y"] \arrow[dr, "f"'] & & y \arrow[dl, "f"] \\ & x & \end{tikzcd}$$

Concretely, we have to show that $f = f \circ \mathsf{id}_y$, which holds because id is the unit for composition.

Now suppose that we have three objects in $\mathcal{C}/x$, which are given by objects $y_1, y_2, y_3 \in \mathcal{C}$ together with morphisms $f_1 : y_1 \to x$, $f_2 : y_2 \to x$, and $f_3 : y_3 \to x$. We also assume that we have morphisms $g_1 : (y_1, f_1) \to_{\mathcal{C}/x} (y_2, f_2)$ and $g_2 : (y_2, f_2) \to_{\mathcal{C}/x} (y_3, f_3)$. This means that we have morphisms $g_1 : y_1 \to_{\mathcal{C}} y_2$ making the following triangle commute.

$$\begin{tikzcd} y_1 \arrow[rr, "g_1"] \arrow[dr, "f_1"'] & & y_2 \arrow[dl, "f_2"] \\ & x & \end{tikzcd}$$

Similarly, we have $g_2 : y_2 \to_{\mathcal{C}} y_3$ making the following triangle commute.

$$\begin{tikzcd}
y_2 \arrow[rr, "g_2"] \arrow[dr, "f_2"'] & & y_3 \arrow[dl, "f_3"] \\
& x &
\end{tikzcd}$$

Their composition in $\mathcal{C}/x$ is given by $g_1 \cdot g_2$. We need to show that the following diagram commutes.

$$\begin{tikzcd}
y_1 \arrow[r, "g_1"] \arrow[dr, "f_1"'] & y_2 \arrow[r, "g_2"] \arrow[d, "f_2"] & y_2 \arrow[dl, "f_3"] \\
& x &
\end{tikzcd}$$

This diagram commutes, because both triangles commute.

We can make this reasoning concrete by presenting it in an equational style. We need to show that $f_1 = (f_3 \cdot g_1) \cdot g_2$.

$$\begin{aligned}
f_1 &= g_1 \cdot f_2 && \text{since } g_1 \text{ is a morphism in } \mathcal{C}/x \\
&= g_1 \cdot (g_2 \cdot f_3) && \text{since } g_2 \text{ is a morphism in } \mathcal{C}/x \\
&= (g_1 \cdot g_2) \cdot f_3 && \text{associativity}
\end{aligned}$$

Hence, $g_1 \cdot g_2$ satisfies the desired requirement.

We leave it to the reader to verify the category laws.

Example 1.13. Let $\mathcal{C}$ be a category. We define the **arrow category** $\mathcal{C}^{\to}$ as follows.

- The objects of $\mathcal{C}^{\to}$ are triples of $x, y \in \mathcal{C}$ together with $f : x \to y$. We denote such objects by $f : x \to y$.
- Given morphisms $f_1 : x_1 \to y_1$ and $f_2 : x_2 \to y_2$, morphisms $f_1 \to_{\mathcal{C}^{\to}} f_2$ are pairs of morphisms $g : x_1 \to x_2$ and $h : y_1 \to y_2$ making the following square commute.

$$\begin{tikzcd}
x_1 \arrow[r, "g"] \arrow[d, "f_1"'] & x_2 \arrow[d, "f_2"] \\
y_1 \arrow[r, "h"'] & y_2
\end{tikzcd}$$

The identity morphism is given by the following square

$$\begin{tikzcd}
x \arrow[r, "\mathsf{id}_x"] \arrow[d, "f"'] & x \arrow[d, "f"] \\
y \arrow[r, "\mathsf{id}_y"'] & y
\end{tikzcd}$$

To define the composition of morphisms, we assume that we have two commutative squares as indicated below.

$$
\begin{tikzcd}
x_1 \arrow[r, "g_1"] \arrow[d, "f_1"'] & x_2 \arrow[d, "f_2"] \\
y_1 \arrow[r, "h_1"'] & y_2
\end{tikzcd}
\qquad
\begin{tikzcd}
x_2 \arrow[r, "g_2"] \arrow[d, "f_2"'] & x_3 \arrow[d, "f_3"] \\
y_2 \arrow[r, "h_2"'] & y_3
\end{tikzcd}
$$

Their composition is defined as the following square.

$$
\begin{tikzcd}
x_1 \arrow[r, "g_1"] \arrow[d, "f_1"'] & x_2 \arrow[r, "g_2"] \arrow[d, "f_2" description] & x_3 \arrow[d, "f_3"] \\
y_1 \arrow[r, "h_1"'] & y_2 \arrow[r, "h_2"'] & y_3
\end{tikzcd}
$$

We leave it as an exercise to the reader to verify that composition and identity indeed give rise to morphisms and to verify the category laws.

Category	**Notation**	**Objects**	**Morphisms**
Category of sets	Set	Sets	Functions
Category of monoids	Mon	Monoids	Homomorphisms
Category of groups	Group	Groups	Homomorphisms
Unit category	unit	Single element	Single morphism
Opposite category	$\mathcal{C}^{\mathsf{op}}$	Objects in $\mathcal{C}$	Morphisms in $\mathcal{C}$ (reverse direction)
Product category	$\mathcal{C}_1 \times \mathcal{C}_2$	Pairs of objects	Pairs of morphisms
Full subcategory	$\mathsf{Full}(\mathcal{C}, \mathsf{P})$	Objects in $\mathcal{C}$	Morphisms in $\mathcal{C}$
Slice category	$\mathcal{C}/x$	Pairs of $y \in \mathcal{C}$ with $f : y \to x$	Commutative triangles
Arrow category		Triples of $x, y \in \mathcal{C}$ and $f : x \to y$	Commutative squares
Preorder	$\widetilde{(X, \leq)}$	Elements in X	Given by $\leq$
Monoids	$\mathsf{B}(M)$	Single element	Elements of M

Table 1: Examples of categories

1.2 Special Morphisms

Definition 1.14. Let $\mathcal{C}$ be a category, and let $x, y : \mathcal{C}$ be objects. We say that $f : x \to y$ is a **monomorphism** if for all objects $w : \mathcal{C}$ and morphisms $g_1, g_2 : w \to y$ such that $f \circ g_1 = f \circ g_2$, we have $g_1 = g_2$.

Definition 1.15. Let $\mathcal{C}$ be a category, and let $x, y : \mathcal{C}$ be objects. We say that $f : x \to y$ is a **epimorphism** if for all objects $z : \mathcal{C}$ and morphisms $g_1, g_2 : y \to z$ such that $g_1 \circ f = g_2 \circ f$, we have $g_1 = g_2$.

Definition 1.16. Let $\mathcal{C}$ be a category, and let $x, y : \mathcal{C}$ be objects. We say that $f : x \to y$ is an **isomorphism** if there is a morphism $g : y \to x$ such that $f \circ g = \mathsf{id}_y$ and $g \circ f = \mathsf{id}_x$. The collection of isomorphisms is denoted by $x \cong y$.

To get a better understanding of these classes of morphisms, we study them for concrete instances of categories. First, we look at Set.

Proposition 1.17. *A function $f : X \to Y$ of sets is injective if and only if it is a monomorphism in the category* Set.

Proof. Suppose that $f : X \to Y$ is injective, and suppose that we have functions $g_1, g_2 : W \to X$ such that $f \circ g_1 = f \circ g_2$. To show that $g_1 = g_2$, we take $w \in W$, and we must show that $g_1(w) = g_2(w)$. Since f is injective, it suffices to show $f(g_1(w)) = f(g_2(w))$, which holds by the assumption that $g_1 \circ f = g_2 \circ f$.

Next we assume that $f : X \to Y$ is a monomorphism, and we show that it is injective. Suppose that we have $x_1, x_2 \in X$ such that $f(x_1) = f(x_2)$. Note that from x_1, x_2 we get morphisms $\overline{x_1}, \overline{x_2} : \mathbb{1} \to X$ (recall that $\mathbb{1} = \{*\}$):

$$\overline{x_1}(*) = x_1, \quad \overline{x_2}(*) = x_2$$

Note that we have that $f \circ \overline{x_1} = f \circ \overline{x_2}$, since

$$\begin{aligned}(f \circ \overline{x_1})(*) &= f(\overline{x_1}(*) \\ &= f(x_1) \\ &= f(x_2) \\ &= f(\overline{x_2}(*) \\ &= (f \circ \overline{x_1})(*)\end{aligned}$$

Since f is a monomorphism, we must thus have $\overline{x_1} = \overline{x_2}$. Hence,

$$x_1 = \overline{x_1}(*) = \overline{x_2}(* = x_2$$

□

Proposition 1.18. *A function $f : X \to Y$ of sets is surjective if and only if it is an epimorphism in the category* Set.

Proof. Suppose that $f : X \to Y$ is surjective, and suppose that we have functions $g_1, g_2 : Y \to Z$ such that $g_1 \circ f = g_2 \circ f$. We must show that $g_1 = g_2$, so we take an arbitrary $y \in Y$, and we must show $g_1(y) = g_2(y)$. Since f is surjective, we can find $x \in X$ such that $f(x) = y$. We can thus conclude that

$$\begin{aligned} g_1(y) &= g_1(f(x)) \\ &= g_2(f(x)) \\ &= g_2(y) \end{aligned}$$

Here we used our assumption that $g_1 \circ f = g_2 \circ f$.

Next we assume that $f : X \to Y$ is an epimorphism, and we need to show that f is surjective. Let $y \in Y$ be arbitrary. We use classical logic to prove that there is $x \in X$ such that $f(x) = y$, so suppose that for each $x \in X$ we have $f(x) \neq y$. Without loss of generality, we assume that $* \notin Y$. We define a maps $g_1, g_2 : Y \to Y \cup \mathbb{1}$ as follows

$$g_1(y') = y'$$

$$g_2(y') = \begin{cases} y' & \text{if } y \neq y' \\ * & \text{if } y = y' \end{cases}$$

Note that $g_1 \neq g_2$, so to acquire a contradiction, it suffices to show $g_1 \circ f = g_2 \circ f$ since f is an epimorphism. Let $x \in X$. Since $f(x) \neq y$ by assumption, we have

$$g_2(f(x)) = f(x) = g_1(f(x))$$

Hence, f is surjective. □

Proposition 1.19. *A function $f : X \to Y$ of sets is bijective if and only if it is an isomorphism in the category* Set*.*

Proof. Suppose that $f : X \to Y$ is bijective. To define $g : Y \to X$, we note that for each $y \in Y$ there is a unique x such that $f(x) = y$, and we define $g(y)$ to be that x. From the description of g, we can immediately see that $f(g(y)) = y$: $g(y)$ is the unique element with $f(g(y)) = y$. To show that $g(f(x)) = x$, we use that f is injective, so it suffices to show that $f(g(f(x))) = f(x)$. This follows from the fact that we just showed that $f(g(y)) = y$ where we take y to be $f(x)$.

Next we assume that $f : X \to Y$ is an isomorphism, which means that we have a function $g : Y \to X$ such that $g \circ f = \mathsf{id}_X$ and $f \circ g = \mathsf{id}_Y$. To show that f is injective, we assume that we have $x_1, x_2 \in X$ such that $f(x_1) = f(x_2)$. Now we note

$$x_1 = g(f(x_1)) = g(f(x_2)) = x_2$$

To show that f is surjective, we let $y \in Y$ be arbitrary. An inverse image for y is given by $g(y)$, since $f(g(y)) = y$ by assumption. □

For the category of monoids and of groups, we can use the same strategy as for the category of sets. We leave this as an exercise (Exercises 1.11).

Next we look at the opposite category $\mathcal{C}^{\mathsf{op}}$.

Proposition 1.20. *A morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a monomorphism in $\mathcal{C}^{\mathsf{op}}$ if and only if $f : y \to_{\mathcal{C}} x$ is a epimorphism in $\mathcal{C}$.*

Proof. Suppose that $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a monomorphism in $\mathcal{C}^{\mathsf{op}}$. Our goal is to show that $f : y \to_{\mathcal{C}} x$ is an epimorphism in $\mathcal{C}$. Suppose that we have $z : \mathcal{C}$ and $g_1, g_2 : x \to_{\mathcal{C}} z$ such that $g_1 \circ_{\mathcal{C}} f = g_2 \circ_{\mathcal{C}} f$. From g_1 and g_2 we obtain morphisms $g_1, g_2 : z \to_{\mathcal{C}^{\mathsf{op}}} x$. Note the following

$$\begin{aligned} f \circ_{\mathcal{C}^{\mathsf{op}}} g_1 &= g_1 \circ_{\mathcal{C}} f \\ &= g_2 \circ_{\mathcal{C}} f \\ &= f \circ_{\mathcal{C}^{\mathsf{op}}} g_2 \end{aligned}$$

Since f is a monomorphism in $\mathcal{C}^{\mathsf{op}}$, we have $g_1 = g_2$, and thus $f : y \to_{\mathcal{C}} x$ is a epimorphism in $\mathcal{C}$.

Now we assume that $f : y \to_{\mathcal{C}} x$ is an epimorphism in $\mathcal{C}$, and we show that $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a monomorphism in $\mathcal{C}^{\mathsf{op}}$. We take the same steps: we assume that we have we have $z : \mathcal{C}$ and $g_1, g_2 : z \to_{\mathcal{C}^{\mathsf{op}}} x$ such that $f \circ_{\mathcal{C}^{\mathsf{op}}} g_1 = f \circ_{\mathcal{C}^{\mathsf{op}}} g_2$. We get morphisms $g_1, g_2 : x \to_{\mathcal{C}} z$ from g_1 and g_2, and we have the following equalities.

$$\begin{aligned} g_1 \circ_{\mathcal{C}} f &= f \circ_{\mathcal{C}^{\mathsf{op}}} g_1 \\ &= f \circ_{\mathcal{C}^{\mathsf{op}}} g_2 \\ &= g_2 \circ_{\mathcal{C}} f \end{aligned}$$

Since f is an epimorphism in $\mathcal{C}$, we get $g_1 = g_2$, which finishes the proof that f is a monomorphism in $\mathcal{C}^{\mathsf{op}}$. □

Proposition 1.21. *A morphism $f : y \to_x$ is a epimorphism in $\mathcal{C}^{\mathsf{op}}$ if and only if f is a monomorphism in $\mathcal{C}$.*

Proof. Left as an exercise (Exercise 1.8). □

Proposition 1.22. *A morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is an isomorphism in $\mathcal{C}^{\mathsf{op}}$ if and only if $f : y \to_{\mathcal{C}} x$ is an isomorphism in $\mathcal{C}$.*

Proof. Suppose that $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is an isomorphism in $\mathcal{C}^{\mathsf{op}}$. This means that we have $g : y \to_{\mathcal{C}^{\mathsf{op}}} x$ such that $g \circ_{\mathcal{C}^{\mathsf{op}}} f = \mathsf{id}_x$ and $f \circ_{\mathcal{C}^{\mathsf{op}}} g = \mathsf{id}_y$. Note that g gives us a morphism $g : x \to_{\mathcal{C}} y$ such that

$$f \circ_{\mathcal{C}} g = g \circ_{\mathcal{C}^{\mathsf{op}}} f = \mathsf{id}_x$$

$$g \circ_{\mathcal{C}} f = f \circ_{\mathcal{C}^{\mathsf{op}}} g = \mathsf{id}_y$$

Hence, $f : y \to_{\mathcal{C}} x$ is an isomorphism in $\mathcal{C}$.

The converse is proven similarly. If $f : y \to_{\mathcal{C}} x$ is an isomorphism in $\mathcal{C}$, then we have $g : x \to_{\mathcal{C}} y$ such that $f \circ_{\mathcal{C}} g = \mathsf{id}_x$ and $g \circ_{\mathcal{C}} f = \mathsf{id}_y$. Hence, we have morphisms $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ and $g : y \to_{\mathcal{C}^{\mathsf{op}}} x$ such that

$$g \circ_{\mathcal{C}^{\mathsf{op}}} f = f \circ_{\mathcal{C}} g = \mathsf{id}_x$$

$$f \circ_{\mathcal{C}^{\mathsf{op}}} g = g \circ_{\mathcal{C}} f = \mathsf{id}_y$$

Hence, f is an isomorphism in $\mathcal{C}^{\mathsf{op}}$. □

The final example that we look at, is $\widetilde{(X, \leq)}$.

Proposition 1.23. *Every morphism in $\widetilde{(X, \leq)}$ is a monomorphism and an epimorphism.*

Proof. To show that some morphism f is a monomorphism, we assume that we have $g_1, g_2 : w \to y$ such that $f \circ g_1 = f \circ g_2$. We must also have $g_1 = g_2$, because all morphisms in $\widetilde{(X, \leq)}$ are equal. Similarly, we can show that every morphism is an epimorphism. $\square$

Proposition 1.24. *We have an isomorphism between x and y in $\widetilde{(X, \leq)}$ if and only if $x \leq y$ and $y \leq x$.*

Proof. An isomorphism $\widetilde{(X, \leq)}$ in x and y consists of morphisms $f : x \to y$ and $g : y \to x$. By definition, these morphisms give us that $x \leq y$ and $y \leq x$. If we have morphisms $x \to y$ if $x \leq y$ and $y \to x$ if $y \leq x$, then we obtain an isomorphism, since we have $f : x \to y$ (since $x \leq y$) and $g : y \to x$ (since $y \leq x$). Because all morphisms are equal in $\widetilde{(X, \leq)}$, the necessary equalities hold, and thus we have an isomorphism. $\square$

Proposition 1.25. *Every isomorphism is a monomorphism and an epimorphism.*

Proof. Left as an exercise (Exercise 1.14). $\square$

Note that the converse of Proposition 1.25 is false.

1.3 Exercises

Exercise 1.1. Complete Example 1.4 by proving that Group is a category.

Exercise 1.2. Finish the proof that $\mathcal{C}^{\mathsf{op}}$ (Example 1.6) forms a category by proving the remain unitality law.

Exercise 1.3. Show that $\mathcal{C}_1 \times \mathcal{C}_2$ forms a category (Example 1.7) by defining the identity and composition of morphisms, and proving the category laws.

Exercise 1.4. Define the category of preorders PreOrd where a morphism from $(X, \leq)$ to $(Y, \leq)$ is a function $f : X \to Y$ such that $f(x) \leq f(y)$ whenever $x \leq y$.

Exercise 1.5. Let $\mathcal{C}$ be a category such that for all $f, g : x \to y$, we have $f = g$. Show that $\mathcal{C}$ induces a preorder on the objects of $\mathcal{C}$.

Exercise 1.6. The objects of the **coslice category** $x/\mathcal{C}$ are pairs of objects $y : \mathcal{C}$ together with $x \to y$. Define the morphisms of the coslice category and show that it indeed forms a category

Exercise 1.7. We consider the **category of families** Fam. The objects of Fam consist of a set X and for each $x \in X$ a set $Y(x)$. Morphisms from (X_1, Y_1) to (X_2, Y_2) consist of a function $f : X_1 \to X_2$ together with maps $g_x : Y_1(x) \to Y_2(f(x))$ for each $x \in X_1$. Show that Fam indeed forms a category.

Exercise 1.8. Prove Proposition 1.21.

Exercise 1.9. Let M be a monoid. Show that M is a group if and only if every morphism in $\mathsf{B}(M)$ is an isomorphism.

Exercise 1.10. Characterize the isomorphism, monomorphisms, and epimorphisms in the product category $\mathcal{C}_1 \times \mathcal{C}_2$.

Exercise 1.11. Characterize the isomorphism, monomorphisms, and epimorphisms in the category Mon of monoids.

Exercise 1.12. Characterize the isomorphism, monomorphisms, and epimorphisms in the category Group of groups.

Exercise 1.13. Give a bijective homorphism of preorders that is not an isomorphism.

Exercise 1.14. Prove Proposition 1.25.

Exercise 1.15. Give a category $\mathcal{C}$ and a morphism f in $\mathcal{C}$ such that f is a monomorphism and an epimorphism, but not an isomorphism.

2 Constructions in Categories

2.1 Terminal and Initial Objects

Definition 2.1. Let $\mathcal{C}$ be a category and let $x \in \mathcal{C}$ be an object. We say that x **is terminal** if for all objects y there is a unique morphism $!_y : y \to x$.

The condition that for all objects y we have a unique morphism $!_y : y \to x$, is called the **universal property of terminal objects**.

Definition 2.2. Let $\mathcal{C}$ be a category and let $x \in \mathcal{C}$ be an object. We say that x **is initial** if for all objects y there is a unique morphism $!_y : x \to y$.

The condition that for all objects y we have a unique morphism $!_y : y \to x$, is called the **universal property of initial objects**.

Example 2.3. The set $\mathbb{1} = \{*\}$ is a terminal object in the category Set of sets. If we have any set X, then we can construct a function $f : X \to \mathbb{1}$, which maps every x to the unique element $*$ of $\mathbb{1}$. Now suppose that we have two functions $f, g : X \to \mathbb{1}$. Our goal is to show that $f = g$. To do so, we take an arbitrary $x \in X$, and we need to show that $f(x) = g(x)$. This follows from the following chain of equalities

$$f(x) = * = g(x)$$

since all elements in $\mathbb{1}$ are equal to $*$.

The empty set $\emptyset$ is an initial object in Set. Let X be any set. We have a function f from $\emptyset \to X$, namely the empty function. In addition, all functions $f, g : \emptyset \to X$ are equal, because there is no $x \in \emptyset$.

Proposition 2.4. *Given an object $x : \mathcal{C}$, x is terminal in $\mathcal{C}$ if and only if x is initial in $\mathcal{C}^{\mathsf{op}}$.*

Proof. Suppose x is terminal in $\mathcal{C}$. To prove that x is initial in $\mathcal{C}^{\mathsf{op}}$, we take any $y : \mathcal{C}^{\mathsf{op}}$, and we need to prove there is a unique morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$. Note that morphisms $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ are the same as morphisms $f : y \to_{\mathcal{C}} x$. Since x is terminal, there is a unique morphism $f : y \to_{\mathcal{C}} x$. Hence, there also is a unique morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ and thus f is initial. In the same way, we prove that x is terminal in $\mathcal{C}$ if x is initial in $\mathcal{C}^{\mathsf{op}}$. □

Proposition 2.5. *Given an object $x : \mathcal{C}$, x is initial in $\mathcal{C}$ if and only if x is terminal in $\mathcal{C}^{\mathsf{op}}$.*

Proof. Exercise. □

Remark 1. Propositions 2.4 and 2.5 capture an important pattern: the notions of initiality and terminality are dual. Terminality corresponds to initiality, but with the morphisms reversed. In category theory, a wide variety of concepts can be dualised, and we will see plenty of examples throughout these notes.

Example 2.6. We construct a terminal and an initial object in Mon. To construct a terminal object in Mon, we note that $\mathbb{1}$ is a monoid. The unit element $e_{\mathbb{1}}$ is $*$ and the multiplication is the function $\cdot_{\mathbb{1}}$ that maps all pairs (x, y) to $*$. Note that every function $f : M \to \mathbb{1}$ is a homomorphism. We have $f(e_M) = * = e_{\mathbb{1}}$ since all elements in $\mathbb{1}$ are equal, and we have

$$f(x \cdot_M y) = * = * \cdot_{\mathbb{1}} * = f(x) \cdot_{\mathbb{1}} f(y)$$

Hence, there is a unique homomorphism from every monoid M to $\mathbb{1}$.

Next we show that $\mathbb{1}$ also is initial in Mon. Let M be any monoid. We define the function $f : \mathbb{1} \to M$ to be $f(*) = e_M$. Note that f is a homomorphism:

$$f(e_{\mathbb{1}}) = f(*) = e_M$$

Since all $x, y : \mathbb{1}$ are equal to $*$, we also have

$$f(x \cdot_{\mathbb{1}} y) = f(* \cdot_{\mathbb{1}} *) = f(*) = e_M$$

$$f(x) \cdot_M f(y) = f(*) \cdot_M f(*) = e_M \cdot_M e_M = e_M$$

Note that f is unique as well. Let g be any other homomorphism from $*$ to M. Then we have

$$g(*) = g(e_{\mathbb{1}}) = e_M = f(*)$$

Hence, $\mathbb{1}$ is both initial and terminal in Mon.

Example 2.7. Let us study initial and terminal objects in the category $\widetilde{(X, \leq)}$ for $(X, \leq)$ be a preorder. A terminal object for $\widetilde{(X, \leq)}$ is an object $x \in X$ such that for each $y \in X$ there is a unique morphism $f : y \to_{\widetilde{(X,\leq)}} x$. Since morphisms in $\widetilde{(X, \leq)}$ are unique and since we have a morphism $f : y \to_{\widetilde{(X,\leq)}} x$ if and only if $y \leq x$, we have that x is a terminal object in $\widetilde{(X, \leq)}$ if and only if for all y we have $y \leq x$. Hence, x is a maximal element of $(X, \leq)$.

An initial object for $\widetilde{(X, \leq)}$ is an object $x \in X$ such that for each $y \in X$ there is a unique morphism $f : x \to_{\widetilde{(X,\leq)}} y$. Hence, x is an initial object in $\widetilde{(X, \leq)}$ if and only if for all y we have $x \leq y$, meaning that x is a minimal element of $(X, \leq)$.

Proposition 2.8. *Let $\mathcal{C}$ be a category and let x and y be terminal objects in $\mathcal{C}$. Then we have a unique isomorphism $f : x \cong y$.*

Proof. Note that we have $f : x \to y$, because y is terminal, and that we have $g : y \to x$, because x is terminal. These morphisms exist due to the "existence" part in the universal property of terminal objects. We have $f \circ g = \mathsf{id}_y$ and $g \circ f = \mathsf{id}_x$ due to the uniqueness part in the universal property of terminal objects. Both $f \circ g$ and id_y are morphisms to the terminal object y, and hence they must be unique. Similarly for $g \circ f = \mathsf{id}_x$. Note that if we have another morphisms $f' : x \to y$, then we have that $f = f'$, because both are morphisms into the terminal object y. □

Proposition 2.9. *Let $\mathcal{C}$ be a category and let x and y be initial objects in $\mathcal{C}$. Then we have a unique isomorphism $f : x \cong y$.*

Proof. Exercise. $\square$

Digression. In Example 1.1 we looked at the simply typed λ-calculus. The objects in the category that we discussed, are types and morphisms are terms. To construct a terminal object in this category, we need to look at a version of the λ-calculus with unit types. Specifically, our λ-calculus needs to come with a type 1 and a term $* : 1$ such that all terms of type 1 are equal. Similarly, we can study initital objects in this category. For those, we need to have a type 0 together with the usual elimination rule for the empty type.

Hence, to obtain a category with a terminal and initial object from the simply typed λ-calculus, we must look at a version that contains the unit type and the empty type.

2.2 Binary Products and Coproducts

Definition 2.10. Suppose that we have a category $\mathcal{C}$ and objects $x, y \in C$. A **binary product for x and y** consists of

- an object $z \in \mathcal{C}$;
- morphisms $\pi_1 : z \to x$ and $\pi_2 : z \to y$;

satisfying the following universal property: for all objects $w \in \mathcal{C}$ and morphisms $f : w \to x$ and $g : w \to y$ there is a unique morphism $\langle f, g\rangle : w \to z$ with

$$\pi_1 \circ \langle f, g\rangle = f$$

$$\pi_2 \circ \langle f, g\rangle = g$$

We call π_1 and π_2 the **projection morphism**, and we call $\langle f, g\rangle$ the **pairing**. We say that $\mathcal{C}$ has **binary products** if all $x, y \in C$ have a binary product. In case $\mathcal{C}$ has binary products, we denote the binary product of x and y by $x \times y$.

To understand Theorem 2.10, we phrase it using the the language of diagrams. We start with z and the projection morphisms.

$$\begin{tikzcd} x & z \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] & y \end{tikzcd}$$

We draw the universal property as follows.

$$\begin{tikzcd} & w \arrow[dl, "f"'] \arrow[d, dashed, "\exists!\langle f,g\rangle"] \arrow[dr, "g"] & \\ x & z \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & y \end{tikzcd}$$

Definition 2.11. Suppose that we have a category $\mathcal{C}$ and objects $x, y \in C$. A **binary coproduct for x and y** consists of

- an object $z \in \mathcal{C}$;
- morphisms $\iota_1 : x \to z$ and $\iota_2 : y \to z$;

satisfying the following universal property: for all objects $w \in \mathcal{C}$ and morphisms $f : x \to w$ and $g : y \to w$ there is a unique morphism $[f, g] : z \to w$ with

$$[f, g] \circ \iota_1 = f$$

$$[f, g] \circ \iota_2 = g$$

We call ι_1 and ι_2 the **inclusion morphisms**, and we call $[f, g]$ the **copairing**. We say that $\mathcal{C}$ has **binary coproducts** if all $x, y \in C$ have a binary coproduct. In case $\mathcal{C}$ has binary coproducts, we denote the binary coproduct of x and y by $x + y$.

We can visualise binary coproducts in a similar way. The inclusion morphisms are drawn as follows.

$$\begin{tikzcd} x \arrow[r, "\iota_1"] & z & y \arrow[l, "\iota_2"'] \end{tikzcd}$$

The universal property is seen as follows.

$$\begin{tikzcd} x \arrow[r, "\iota_1"] \arrow[dr, "f"'] & z \arrow[d, dashed, "{\exists ! [f,g]}"] & y \arrow[l, "\iota_2"'] \arrow[dl, "g"] \\ & w & \end{tikzcd}$$

Example 2.12. We show that the category Set of sets has binary products. Let X and Y be sets, and we define the set $X \times Y$ as follows

$$X \times Y = \{(x, y) \mid x \in X, y \in Y\}$$

We claim that $X \times Y$ is the binary product of X and Y. To see why, we define the following morphisms. We start with $\pi_1 : X \times Y \to X$:

$$\pi_1(x, y) = x$$

The morphism $\pi_2 : X \times Y \to Y$ is defined similarly:

$$\pi_2(x, y) = y$$

Next we need to prove the universal property. Let W be any set and suppose that we have functions $f : W \to X$ and $g : W \to Y$. We define $\langle f, g\rangle : W \to X \times Y$ as follows:

$$\langle f, g\rangle(w) = (f(w), g(w))$$

Note that $\pi_1 \circ \langle f, g\rangle = f$, because for every $w \in W$ we have

$$(\pi_1 \circ \langle f, g\rangle)(w) = \pi_1(\langle f, g\rangle(w)) = \pi_1(f(w), g(w)) = f(w)$$

Similarly we prove that $\pi_2 \circ \langle f, g \rangle = g$.

$$(\pi_2 \circ \langle f, g\rangle)(w) = \pi_2(\langle f, g\rangle(w)) = \pi_2(f(w), g(w)) = g(w)$$

We also need to show that $\langle f, g\rangle$ is the unique morphism with this property, so we assume that we have some $h : W \to X \times Y$ such that $\pi_1 \circ h = f$ and $\pi_2 \circ h = g$. Let $w \in W$ be arbitrary, and note that

$$\langle f, g\rangle(w) = (f(w), g(w)) = (\pi_1(h(w)), \pi_2(h(w))) = h(w)$$

Hence, we have $\langle f, g\rangle = h$, meaning that $\langle f, g\rangle$ is the unique morphism with this property. This proves that $X \times Y$ is indeed the binary product of X and Y.

Example 2.13. Next we show that the category Set has binary coproducts. Given sets X and Y, we define the set $X + Y$ as follows

$$X + Y = \{(0, x) \mid x \in X\} \cup \{(1, y) \mid y \in Y\}$$

We call $X + Y$ the **disjoint union** of X and Y. We show that $X + Y$ is the coproduct of X and Y. We start by defining the morphism $\iota_1 : X \to X + Y$:

$$\iota_1(x) = (0, x)$$

We define $\iota_2 : Y \to X + Y$ similarly:

$$\iota_2(y) = (1, y)$$

Now we need to prove the universal property, so we take an arbitrary set W and functions $f : X \to W$ and $g : Y \to W$. We define $[f, g] : X + Y \to W$ as follows

$$[f, g](z) = \begin{cases} f(x) & \text{if } z = (0, x) \\ g(y) & \text{if } z = (1, y) \end{cases}$$

Note that $[f, g]$ is total, because all elements in $X + Y$ are either of the form $(0, x)$ or $(1, y)$. We can directly prove the following equations

$$[f, g](\iota_1(x)) = [f, g](0, x) = f(x)$$

$$[f, g](\iota_2(y)) = [f, g](1, y) = g(y)$$

To prove the uniqueness requirement, we assume that we have some $h : X+Y \to W$ such that $h \circ \iota_1 = f$ and $h \circ \iota_2 = g$. If we have $z \in X + Y$, then we know that z is either of the form $(0, x)$ or of the form $(1, y)$ where $x \in X$ or $y \in Y$. In the first case, we have

$$h(0, x) = h(\iota_1(x)) = f(x) = [f, g](0, x) = [f, g](\iota_1(x))$$

In the second case, we have

$$h(1, y) = h(\iota_2(y)) = f(y) = [f, g](1, y) = [f, g](\iota_2(y))$$

This concludes the proof of the uniqueness requirement, and hence $X + Y$ is indeed a binary coproduct of X and Y.

Proposition 2.14. *Let $\mathcal{C}$ be a category and suppose that we have objects $x, y, z \in \mathcal{C}$. Then z is a binary product of x and y in $\mathcal{C}$ if and only if z is a binary coproduct of x and y in $\mathcal{C}^{\mathsf{op}}$. Dually, z is a binary coproduct of x and y in $\mathcal{C}$ if and only if z is a binary product of x and y in $\mathcal{C}^{\mathsf{op}}$.*

Proof. Exercise □

Example 2.15. We show that the category Mon has binary products. Let M and N be monoids. We first show that $M \times N = \{(m, n) \mid m \in M, n \in N\}$ is a monoid. We define $e_{M\times N}$ to be (e_M, e_N), and we define $(m_1, n_1) \cdot_{M\times N} (m_2, n_2)$ to be $(m_1 \cdot m_2, n_1 \cdot n_2)$. To show that $M \times N$ is the binary product of M and N, we first note that π_1 and π_2 are homomorphisms. For π_1, this is so, because

$$\pi_1(e_{M\times N}) = \pi_1(e_M, e_N) = e_M$$

$$\begin{aligned}\pi_1((m_1, n_1) \cdot_{M\times N} (m_2, n_2)) &= \pi_1(m_1 \cdot m_2, n_1 \cdot n_2)\\ &= m_1 \cdot m_2\\ &= \pi_1(m_1, n_1) \cdot \pi_1(m_2, n_2)\end{aligned}$$

We can use the same reasoning for π_2. If $f : W \to M$ and $g : W \to N$ are homomorphisms, then so is $\langle f, g\rangle$. This is because

$$\begin{aligned}\langle f, g\rangle(e_W) &= (f(e_W), g(e_W))\\ &= (e_M, e_N)\\ &= e_{M\times N}\end{aligned}$$

$$\begin{aligned}\langle f, g\rangle(w_1 \cdot w_2) &= (f(w_1 \cdot w_2), g(w_1 \cdot w_2))\\ &= (f(w_1) \cdot f(w_2), g(w_1) \cdot g(w_2))\\ &= (f(w_1), g(w_1)) \cdot (f(w_2), g(w_2))\\ &= \langle f, g\rangle(w_1) \cdot \langle f, g\rangle(w_2)\end{aligned}$$

The uniqueness of $\langle f, g\rangle$ is proven in the same way as for Set.

Constructing binary coproducts in the category of monoids is more challenging, and we leave it as an exercise.

Example 2.16. Let $(X, \leq)$ be a preorder. We look at binary products in $\widetilde{(X, \leq)}$. Given $x, y \in X$, a binary product for x and y is an element $z \in X$ such that $x \leq z$ and $y \leq z$. This z also satisfies another property: for every $w \in X$ with $x \leq w$ and $y \leq w$, we must also have $z \leq w$. Hence, z is a least upper bound of x and y.

We can similarly show that binary coproducts of x and y are greatest lower bounds. Given $x, y \in X$, a binary coproduct for x and y is an element $z \in X$ such that $z \leq x$ and $z \leq y$ such that for all $w \in X$ with $w \leq x$ and $w \leq y$, we have $w \leq z$.

Proposition 2.17. *Let $\mathcal{C}$ be a category and let x and y be objects in $\mathcal{C}$. Suppose that $z_1, z_2 : \mathcal{C}$ are binary products of x and y. Then we have an isomorphism $f : z_1 \cong z_2$.*

Proof. Since z_1 is a binary product of x and y, we have morphisms $\pi_1 : z_1 \to x$ and $\pi_2 : z_2 \to y$. Similarly, we have morphisms $\rho_1 : z_2 \to x$ and $\rho_2 : z_2 \to y$. We first construct a morphism $f : z_1 \to z_2$, which we define to be $\langle \rho_1, \rho_2 \rangle$ as displayed below.

$$\begin{tikzcd}
& z_1 \arrow[dl, "\pi_1"'] \arrow[dr, "\pi_2"] \arrow[dd, dashed, "\exists!\langle\rho_1,\rho_2\rangle" description] & \\
x & & y \\
& z_2 \arrow[ul, "\rho_1"] \arrow[ur, "\rho_2"'] &
\end{tikzcd}$$

We define $g : z_2 \to z_1$ to be $\langle \pi_1, \pi_2 \rangle$.

$$\begin{tikzcd}
& z_1 \arrow[dl, "\pi_1"'] \arrow[dr, "\pi_2"] & \\
x & & y \\
& z_2 \arrow[ul, "\rho_1"] \arrow[ur, "\rho_2"'] \arrow[uu, dashed, "\exists!\langle\pi_1,\pi_2\rangle" description] &
\end{tikzcd}$$

We want to prove that $f \circ g = \mathsf{id}_{z_2}$. To do so, it suffices to prove that $f \circ g \circ \rho_1 = \mathsf{id}_{z_2} \circ \rho_1$ and that $f \circ g \circ \pi_2 = \mathsf{id}_{z_2} \circ \rho_2$. This follows from the following equalities

$$\begin{aligned}
f \circ g \circ \rho_1 &= f \circ \langle \pi_1, \pi_2 \rangle \circ \rho_1 \\
&= f \circ \pi_1 \\
&= \langle \rho_1, \rho_2 \rangle \circ \pi_1 \\
&= \rho_1
\end{aligned}$$

$$\begin{aligned}
f \circ g \circ \rho_2 &= f \circ \langle \pi_1, \pi_2 \rangle \circ \rho_2 \\
&= f \circ \pi_2 \\
&= \langle \rho_1, \rho_2 \rangle \circ \pi_2 \\
&= \rho_2
\end{aligned}$$

We prove that $g \circ f = \mathsf{id}_{z_1}$ in the same way. ☐

Proposition 2.18. *Let $\mathcal{C}$ be a category and let x and y be objects in $\mathcal{C}$. Suppose that $z_1, z_2 : \mathcal{C}$ are binary coproducts of x and y. Then we have an isomorphism $f : z_1 \cong z_2$.*

Proof. Exercise. ☐

Example 2.19. We introduce the following notation. Let $\mathcal{C}$ be a category with binary products. Given $f : x_1 \to x_2$ and $g : y_1 \to y_2$, we define $f \times g : x_1 \times y_1 \to x_2 \times y_2$ as follows.

$$\begin{tikzcd}
x_1 \arrow[d, "f"'] & x_1 \times y_1 \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, dashed, "f\times g"] & y_1 \arrow[d, "g"] \\
x_2 & x_2 \times y_2 \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & y_2
\end{tikzcd}$$

Note we have that $\pi_1 \circ (f \times g) = f \circ \pi_1$ and that $\pi_2 \circ (f \times g) = g \circ \pi_2$. Dually, if $\mathcal{C}$ is a category with binary coproducts, and if we have morphisms $f : x_1 \to x_2$ and $g : y_1 \to y_2$, we define $f + g : x_1 + y_1 \to x_2 + y_2$ as follows.

$$\begin{tikzcd}
x_1 \arrow[d, "f"'] \arrow[r, "\iota_1"] & x_1 + y_1 \arrow[d, dashed, "f+g"] & y_1 \arrow[l, "\iota_2"'] \arrow[d, "g"] \\
x_2 \arrow[r, "\iota_1"'] & x_2 + y_2 & y_2 \arrow[l, "\iota_2"]
\end{tikzcd}$$

We have

$$(f + g) \circ \iota_1 = \iota_1 \circ f$$
$$(f + g) \circ \iota_2 = \iota_2 \circ g$$

Digression. To guarantee that the category defined in Example 1.1 has binary products and binary coproducts, we again need to consider suitable extensions to the type systems. If we want this category to have binary products, we must have a type constructor that all types τ and σ and gives us a type $\tau \times \sigma$. For the type $\tau \times \sigma$, we need to have a first and second projection, but also a pairing operation that allows us to construct terms of type $\tau\times\sigma$. In addition, these terms need to satisfy the relevant β- and η-rules: η rules guarantee uniqueness, whereas β-rules give that the required diagrams commute. For binary coproducts, we need to extend the type system with coproduct types. The extension is dual: we must have types $\tau + \sigma$, left and right inclusion for terms, and a term that allows us to do case distinction. We need to require β- and η-rules to guarantee uniqueness and that the required diagrams commute.

2.3 Pullbacks

Definition 2.20. Let $\mathcal{C}$ be a category with objects $x, y, z \in C$. Suppose we have morphisms $f : x \to z$ and $g : y \to z$. A **pullback** for f and g consists of

- an object $w \in \mathcal{C}$
- morphisms $\pi_1 : w \to x$ and $\pi_2 : w \to y$

such that $f \circ \pi_1 = g \circ \pi_2$. We require the following universal property: for all objects $a \in \mathcal{C}$ with morphisms $h_1 : a \to x$ and $h_2 : a \to y$ such that $h_1 \cdot f = h_2 \cdot g$, there is a unique $k : a \to w$ such that $k \cdot \pi_1 = f$ and $k \cdot \pi_2 = g$.

We say that $\mathcal{C}$ has pullbacks if every f and g has a pullback.

To understand the definition of pullbacks, we visualise the definition using diagrams. We start with the following morphisms.

$$\begin{tikzcd}
 & y \arrow[d, "g"] \\
x \arrow[r, "f"'] & z
\end{tikzcd}$$

Our goal is to say that w is a pullback of f and g, which means that we have a commuting diagram as follows.

$$\begin{tikzcd}
w \arrow[r, "\pi_2"] \arrow[d, "\pi_1"'] \arrow[dr, phantom, "\lrcorner", very near start] & y \arrow[d, "g"] \\
x \arrow[r, "f"'] & z
\end{tikzcd}$$

The symbol $\lrcorner$ indicates that this square is a pullback, which means that the following universal property is satisfied.

$$\begin{tikzcd}
a \arrow[dr, dashed, "\exists! k"] \arrow[drr, bend left, "h_2"] \arrow[ddr, bend right, "h_1"'] & & \\
 & w \arrow[r, "\pi_2"] \arrow[d, "\pi_1"'] \arrow[dr, phantom, "\lrcorner", very near start] & y \arrow[d, "g"] \\
 & x \arrow[r, "f"'] & z
\end{tikzcd}$$

Specifically, we assume that we have $h_1 : a \to x$ and $h_2 : a \to y$ making the diagram below commute.

$$\begin{tikzcd}
a \arrow[r, "h_2"] \arrow[d, "h_1"'] & y \arrow[d, "g"] \\
x \arrow[r, "f"'] & z
\end{tikzcd}$$

Then we obtain $k : a \to w$ making the triangles below commute

$$\begin{tikzcd}
 & a \arrow[dl, "h_1"'] \arrow[d, "k"] \arrow[dr, "h_2"] & \\
x & w \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & y
\end{tikzcd}$$

Example 2.21. Let X, Y, Z be sets and suppose that we have functions $f : X \to Z$ and $g : Y \to Z$. The pullback for f and g is the following set

$$W = \{(x, y) \mid x \in X, y \in Y, f(x) = g(y)\}$$

To show that W is indeed the pullback of f and g, we define the following functions. We start with $\pi_1 : W \to X$.

$$\pi_1(x, y) = x$$

Next we define $\pi_2 : W \to Y$.

$$\pi_2(x, y) = y$$

Note that $f \circ \pi_1 = g \circ \pi_2$. Every element of W is of the form (x, y) with $x \in X$, $y \in Y$ and $f(x) = g(y)$. Hence,

$$(f \circ \pi_1)(x, y) = f(\pi_1(x, y)) = f(x) = g(y) = g(\pi_2(x, y)) = (g \circ \pi_2)(x, y)$$

Now suppose that we have some set A with functions $h_1 : A \to X$ and $h_2 : A \to Y$ such that $h_1 \circ \pi_1 = f$ and $h_2 \circ \pi_2 = g$. We define a map $k : A \to W$ as follows

$$k(a) = (h_1(a), h_2(a))$$

Note that $k(a)$ is indeed an element of W since $f(h_1(a)) = g(h_2(a))$ by assumption. In addition,

$$(\pi_1 \circ k)(a) = \pi_1(k(a)) = \pi_1(h_1(a), h_2(a)) = h_1(a)$$

$$(\pi_2 \circ k)(a) = \pi_2(k(a)) = \pi_2(h_1(a), h_2(a)) = h_2(a)$$

To prove that k is unique, we assume that we have $k' : A \to W$ such that $\pi_1 \circ k' = h_1$ and $\pi_2 \circ k' = h_2$. Then we must also have for every $a \in A$

$$k(a) = (h_1(a), h_2(a)) = (\pi_1(k'(a)), \pi_2(k'(a))) = k'(a)$$

Hence, W is indeed a pullback.

Example 2.22. Let $(X, \leq)$ be a preorder. Pullbacks in $\widetilde{(X, \leq)}$ are the same as least upper bounds. If we have $x \leq z$ and $y \leq z$, then a pullback consists of $w \in X$ such that for all $a \in X$ with $a \leq x$ and $a \leq y$, we must also have $a \leq w$. Hence, w is the least upper bound of x and y.

Proposition 2.23. *Pullbacks are unique up to isomorphism.*

Proof. Exercise. □

2.4 Exercises

Exercise 2.1. Prove Proposition 2.5.

Exercise 2.2. Prove Proposition 2.9.

Exercise 2.3. Prove Proposition 2.14.

Exercise 2.4. Let $\mathcal{C}$ be a category with binary products. Construct an isomorphism between $x \times y$ and $y \times x$.

Exercise 2.5. Let $\mathcal{C}$ be a category with binary coproducts. Construct an isomorphism between $x + (y + z)$ and $(x + y) + z$.

Exercise 2.6. Let $\mathcal{C}$ be a category with binary products and a terminal object $\mathbb{1}$. Construct an isomorphism between $x \times \mathbb{1}$ and x.

Exercise 2.7. Let $\mathcal{C}$ be a category with binary coproducts and an initial object $\mathbb{0}$. Construct an isomorphism between $x + \mathbb{0}$ and x.

Exercise 2.8. Let $\mathcal{C}$ be a category with a terminal object $\mathbb{1}$. Show that an object $x : \mathcal{C}$ is terminal whenever we have an isomorphism $f : x \cong \mathbb{1}$.

Exercise 2.9. Let $\mathcal{C}$ be a category with an initial object $\mathbb{0}$. Show that an object $x : \mathcal{C}$ is initial whenever we have an isomorphism $f : x \cong \mathbb{1}$.

Exercise 2.10. Let $\mathcal{C}$ be a category and let z be a binary product of x and y. Show that an object $w : \mathcal{C}$ is a binary product of x and y as well whenever we have an isomorphism $f : w \cong z$.

Exercise 2.11. Let $\mathcal{C}$ be a category and let z be a binary coproduct of x and y. Show that an object $w : \mathcal{C}$ is a binary product of x and y as well whenever we have an isomorphism $f : w \cong z$.

Exercise 2.12. Let $\mathcal{C}$ be a category with a terminal object $\mathbb{1}$. Show that every morphism $f : \mathbb{1} \to x$ is a monomorphism.

Exercise 2.13. Let $\mathcal{C}$ be a category with an initial object $\mathbb{0}$. Show that every morphism $f : x \to \mathbb{0}$ is an epimorphism.

Exercise 2.14. Let $\mathcal{C}$ be a category with binary products and a terminal object. Show that the following square is a pullback.

$$\begin{tikzcd}
x \times y \arrow[r, "\pi_2"] \arrow[d, "\pi_1"'] & y \arrow[d, "!_y"] \\
x \arrow[r, "!_y"'] & \mathbb{1}
\end{tikzcd}$$

Exercise 2.15. Let $\mathcal{C}$ be a category with binary products. Prove that the following square is a pullback.

$$\begin{tikzcd}
x_1 \times y \arrow[r, "f \times \mathsf{id}_y"] \arrow[d, "\pi_1"'] & x_2 \times y \arrow[d, "\pi_1"] \\
x_1 \arrow[r, "f"'] & x_2
\end{tikzcd}$$

Exercise 2.16. Prove Proposition 2.23

Exercise 2.17. Give the dual notion of pullbacks, called **pushout**.

3 Functors and Natural Transformations

3.1 Functors

Definition 3.1. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be categories. A **functor** $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ consists of

- for each object $x : \mathcal{C}_1$ an object $F(x) : \mathcal{C}_2$;
- for each morphism $f : x \to_{\mathcal{C}_1} y$ a morphism $F(f) : F(x) \to_{\mathcal{C}_2} F(y)$

such that

$$\mathsf{F}(\mathsf{id}_x) = \mathsf{id}_{F(x)}$$

and for all $f : x \to_{\mathcal{C}_1} y$ and $g : y \to_{\mathcal{C}_1} z$ we have

$$F(g \circ f) = F(g) \circ F(f).$$

Example 3.2. Let $\mathcal{C}$ be a category. We define the **identity functor** $\mathsf{Id}_{\mathcal{C}} : \mathcal{C} \to \mathcal{C}$ as follows.

- $\mathsf{Id}_{\mathcal{C}}(x) = x$ for $x : \mathcal{C}$
- $\mathsf{Id}_{\mathcal{C}}(f) = f$ for $f : x \to y$

Note that we indeed obtain a functor, because

$$\mathsf{Id}_{\mathcal{C}}(\mathsf{id}_x) = \mathsf{id}_x = \mathsf{id}_{\mathsf{Id}_{\mathcal{C}}(x)}$$

$$\mathsf{Id}_{\mathcal{C}}(g \circ f) = g \circ f = \mathsf{Id}_{\mathcal{C}}(g) \circ \mathsf{Id}_{\mathcal{C}}(f)$$

Example 3.3. Let $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{C}_3$ be functors. We define the **composition functor** $\mathsf{G} \circ \mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_3$ as follows.

- $(\mathsf{G} \circ \mathsf{F})(x) = \mathsf{G}(\mathsf{F}(x))$
- $(\mathsf{G} \circ \mathsf{F})(f) = \mathsf{G}(\mathsf{F}(f))$

We indeed obtain a functor, since we have

$$(\mathsf{G} \circ \mathsf{F})(\mathsf{id}_x) = \mathsf{G}(\mathsf{F}(\mathsf{id}_x)) = \mathsf{G}(\mathsf{id}_{\mathsf{F}(x)}) = \mathsf{id}_{\mathsf{G}(\mathsf{F}(x))}$$

$$(\mathsf{G} \circ \mathsf{F})(g \circ f) = \mathsf{G}(\mathsf{F}(g \circ f)) = \mathsf{G}(\mathsf{F}(g) \circ \mathsf{F}(f)) = \mathsf{G}(\mathsf{F}(g)) \circ \mathsf{G}(\mathsf{F}(f))$$

Example 3.4. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be categories, and let $y : \mathcal{C}_2$ be an object. We define the **constant functor** $\mathsf{Const}_y : \mathcal{C}_1 \to \mathcal{C}_2$ as follows.

- $\mathsf{Const}_y(x) = y$
- $\mathsf{Const}_y(f) = \mathsf{id}_y$

This is indeed a functor, because

$$\mathsf{Const}_y(\mathsf{id}_x) = \mathsf{id}_y$$

$$\mathsf{Const}_y(g \circ f) = \mathsf{id}_y = \mathsf{id}_y \circ \mathsf{id}_y = \mathsf{Const}_y(g) \circ \mathsf{Const}_y(f)$$

Recall that given sets X and Y, we define their **disjoint union** (Example 2.13) to be

$$X + Y = \{(0, x) \mid x \in X\} \cup \{(1, y) \mid y \in Y\}$$

If we have functions $f : X_1 \to X_2$ and $g : Y_1 \to Y_2$, then we define $f + g : X_1 + Y_2 \to X_2 + Y_2$ as follows.

$$(f+g)(w) = \begin{cases} (0, f(x)) & \text{if } w = (0, x) \text{ for some } x \in X_1 \\ (1, g(y)) & \text{if } w = (1, y) \text{ for some } y \in Y_1 \end{cases}$$

Example 3.5. We define the functor $\mathsf{Maybe} : \mathsf{Set} \to \mathsf{Set}$, which we call the **maybe functor** as follows.

- $\mathsf{Maybe}(X) = X + \mathbb{1}$
- $\mathsf{Maybe}(f) = f + \mathsf{id}_{\mathbb{1}}$

If we have $f : X_1 \to X_2$, then $\mathsf{Maybe}(f)$ is defined to be $f + \mathsf{id}_{\mathbb{1}} : X_1 + \mathbb{1} \to X_2 + \mathbb{1}$. Hence, $\mathsf{Maybe}(f)$ is indeed a morphism from $\mathsf{Maybe}(X_1) \to \mathsf{Maybe}(X_2)$. To show that Maybe is a functor, we prove the functor laws. First, we show that Maybe preserves the identity. Let X be any set and let $z : X + \mathbb{1}$ be any element. We need to show that $(\mathsf{id}_X + \mathsf{id}_{\mathbb{1}})(z) = z$. There are two cases. If $z = (0, x)$ for $x \in X$, then we have

$$(\mathsf{id}_X + \mathsf{id}_{\mathbb{1}})(0, x) = (0, \mathsf{id}_X(x)) = (0, x)$$

Otherwise, we have $z = (1, *)$, and then we have

$$(\mathsf{id}_X + \mathsf{id}_{\mathbb{1}})(1, *) = (1, \mathsf{id}_{\mathbb{1}}(*)) = (1, *)$$

Second, we show that Maybe preserves composition. Suppose that we have function $f : X \to Y$ and $g : Y \to Z$. To show that $(g \circ f) + \mathsf{id}_{\mathbb{1}} = (g + \mathsf{id}_{\mathbb{1}}) \circ (f + \mathsf{id}_{\mathbb{1}})$. Let $w : X + \mathbb{1}$ be arbitrary. Again there are two cases. If $w = (0, x)$ for some $x \in X$, we have

$$\begin{aligned} ((g \circ f) + \mathsf{id}_{\mathbb{1}})(0, x) &= (0, (g \circ f)(x)) \\ &= (0, g(f(x))) \\ &= (g + \mathsf{id}_{\mathbb{1}})(0, f(x)) \\ &= (g + \mathsf{id}_{\mathbb{1}})((f + \mathsf{id}_{\mathbb{1}})(0, x)) \\ &= ((g + \mathsf{id}_{\mathbb{1}}) \circ (f + \mathsf{id}_{\mathbb{1}}))(0, x) \end{aligned}$$

Otherwise, it must be that $w = (1, *)$, and in that case we have

$$\begin{aligned} ((g \circ f) + \mathsf{id}_{\mathbb{1}})(1, *) &= (1, \mathsf{id}_{\mathbb{1}}(*)) \\ &= (g + \mathsf{id}_{\mathbb{1}})(1, *) \\ &= (g + \mathsf{id}_{\mathbb{1}})((f + \mathsf{id}_{\mathbb{1}})(1, *)) \\ &= ((g + \mathsf{id}_{\mathbb{1}}) \circ (f + \mathsf{id}_{\mathbb{1}}))(1, *) \end{aligned}$$

Digression. The functor Maybe is important in functional programming. Programming languages like Haskell are designed to be pure by default. However, for various purposes, one would like to have effectful computations, for instance to deal with input and output, partial computations, and computations that might not terminate. Monads are used in Haskell for these purposes. We discuss monads in more detail in Section 6, but for now it only is important to know that every monad comes with a functor. To deal with partial computation, one uses the Maybe monad in Haskell, and the definition of the functor underlying the Maybe monad is exactly the functor we defined in Example 3.5. The reason why this functor is important for partial computations is as follows: functions $f : X \to Y + \mathbb{1}$ are the same as partial functions from X to Y.

If we have functions $f : X_1 \to X_2$ and $g : Y_1 \to Y_2$ between sets, then we also have a function $f \times g : X_1 \times Y_1 \to X_2 \times Y_2$ defined as follows

$$(f \times g)(x, y) = (f(x), g(y))$$

Note that we can obtain this function using Examples 2.12 and 2.19.

Example 3.6. Let A be any set. We define the functor $(-) \times A : \mathsf{Set} \to \mathsf{Set}$ as follows.

- $((-) \times A)(X) = X \times A$
- $((-) \times A)(f) = f \times \mathsf{id}_A$

We show that $(-) \times A$ is a functor. First, we show that $(-) \times A$ preserves the identity. To do so, we need to show that $(\mathsf{id}_X \times \mathsf{id}_A) = \mathsf{id}_{X \times A}$, so we take an arbitrary $z \in X \times A$. Since we have $z = (x, a)$ with $x \in X$ and $a \in A$, we also have

$$\begin{aligned}
(\mathsf{id}_X \times \mathsf{id}_A)(x, a) &= (\mathsf{id}_X(x), \mathsf{id}_A(a)) \\
&= (x, a) \\
&= \mathsf{id}_{X \times A}(x, a)
\end{aligned}$$

Second, we show that $(-) \times A$ preserves composition, so we need to show that $(g \circ f) \times \mathsf{id}_A = (g \times \mathsf{id}_A) \circ (f \times \mathsf{id}_A)$, where $f : X \to Y$ and $g : Y \to Z$. If we have any $x \in X$ and $a \in A$, then we also have

$$\begin{aligned}
((g \circ f) \times \mathsf{id}_A)(x, a) &= ((g \circ f)(x), \mathsf{id}_A(a)) \\
&= (g(f(x)), a) \\
&= (g(f(x)), \mathsf{id}_A(a)) \\
&= (g \times \mathsf{id}_A)(f(x), a) \\
&= (g \times \mathsf{id}_A)(f(x), \mathsf{id}_A(a)) \\
&= (g \times \mathsf{id}_A)((f \times \mathsf{id}_A)(x, a)) \\
&= ((g \times \mathsf{id}_A) \circ (f \times \mathsf{id}_A))(x, a)
\end{aligned}$$

Example 3.7. Let A be any set. We have a functor $\mathcal{L}_A : \mathsf{Set} \to \mathsf{Set}$. Given a set X, we define $\mathcal{L}_A(X)$ to be $X \times A + \mathbb{1}$. Given a function $f : X \to Y$, we define $\mathcal{L}_A(f) : X \times A + \mathbb{1} \to Y \times A + \mathbb{1}$ as follows

$$\mathcal{L}_A(f)(w) = \begin{cases} (0, (f(x), a)) & \text{if } w = (0, (x, a)) \\ (1, *) & \text{if } w = (1, *) \end{cases}$$

We leave it as an exercise to prove that this is indeed a functor.

Digression. Example 3.7 turns up in Section 4. In that section, we study algebras and coalgebras for functors, which are used to study inductive and coinductive data types in functional programming languages like Haskell. We show that we can use the functor $\mathcal{L}_A$ to describe the data type of lists.

Example 3.8. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G} : \mathcal{D}_1 \to \mathcal{D}_2$. We define a functor $\mathsf{F} \times \mathsf{G} : \mathcal{C}_1 \times \mathcal{D}_1 \to \mathcal{C}_2 \times \mathcal{D}_2$ as follows.

- $(\mathsf{F} \times \mathsf{G})(x, y) = (\mathsf{F}(x), \mathsf{G}(y))$
- $(\mathsf{F} \times \mathsf{G})(f, g) = (\mathsf{F}(f), \mathsf{G}(g))$

We leave as an exercise to verify that $\mathsf{F} \times \mathsf{G}$ is indeed a functor.

Example 3.9. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{D}$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{D}$, and suppose that $\mathcal{D}$ has binary products. Given objects $x, y \in D$, we denote their binary product by $x \times_{\mathcal{D}} y$. We define the functor $\mathsf{F} \times_{\mathcal{D}} \mathsf{G} : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{D}$ as follows.

- $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(x, y) = \mathsf{F}(x) \times_{\mathcal{D}} G(y)$
- given $f : x_1 \to x_2$ and $g : y_1 \to y_2$ we define $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f, g) : \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) \to \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2)$ to be $F(f) \times \mathsf{G}(g)$

To visualise the action on morphisms, we use the following diagram

$$\begin{tikzcd}
\mathsf{F}(x_1) \arrow[d, "\mathsf{F}(f)"'] & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, dashed, "F(f) \times \mathsf{G}(g)"] & \mathsf{G}(y_1) \arrow[d, "\mathsf{G}(g)"] \\
\mathsf{F}(x_2) & \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathsf{G}(y_2)
\end{tikzcd}$$

By definition, $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f, g)$ is the unique morphism making this diagram commute. To show that $\mathsf{F} \times_{\mathcal{D}} \mathsf{G}$ is a functor, we prove the functor laws. We first show that $\mathsf{F} \times_{\mathcal{D}} \mathsf{G}$ preserves the identity. To show that $\mathsf{F}(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y) = \mathsf{id}_{\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y)}$, we note that $\mathsf{F}(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y)$ is the unique morphism making the following diagram commute.

$$\begin{tikzcd}
\mathsf{F}(x) \arrow[d, "\mathsf{F}(\mathsf{id}_x)"'] & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, dashed, "F(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y)"] & \mathsf{G}(y) \arrow[d, "\mathsf{G}(\mathsf{id}_y)"] \\
\mathsf{F}(x) & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathsf{G}(y)
\end{tikzcd}$$

Hence, we get that $\mathsf{F}(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y) = \mathsf{id}_{\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y)}$ from the fact the following diagram commutes.

$$\begin{tikzcd}
\mathsf{F}(x) \arrow[d, "\mathsf{F}(\mathsf{id}_x)"'] & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, "\mathsf{id}_{\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y)}"] & \mathsf{G}(y) \arrow[d, "\mathsf{G}(\mathsf{id}_y)"] \\
\mathsf{F}(x) & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathsf{G}(y)
\end{tikzcd}$$

Next we show that $\mathsf{F} \times_{\mathcal{D}} \mathsf{G}$ preserves composition. Suppose that we have morphisms $f_1 : x_1 \to x_2$ and $f_2 : x_2 \to x_3$ in $\mathcal{C}_1$, and morphisms $g_1 : y_1 \to y_2$ and $g_2 : y_2 \to y_3$ in $\mathcal{C}_2$. The morphism $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2 \circ f_1, g_2 \circ g_1)$ is defined to be $F(f_2 \circ f_1) \times \mathsf{G}(g_2 \circ g_1)$, which is the unique morphism making the following diagram commutes.

$$\begin{tikzcd}
\mathsf{F}(x_1) \arrow[d, "\mathsf{F}(f_2 \circ f_1)"'] & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, "F(f_2 \circ f_1) \times \mathsf{G}(g_2 \circ g_1)"] & \mathsf{G}(y_1) \arrow[d, "\mathsf{G}(g_2 \circ g_1)"] \\
\mathsf{F}(x_3) & \mathsf{F}(x_3) \times_{\mathcal{D}} \mathsf{G}(y_3) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathsf{G}(y_3)
\end{tikzcd}$$

We visualise $((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2, g_2)) \circ ((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_1, g_1))$ using the following diagram.

$$\begin{tikzcd}
\mathsf{F}(x_1) \arrow[d, "\mathsf{F}(f_1)"'] & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, "F(f_1) \times \mathsf{G}(g_1)"] & \mathsf{G}(y_1) \arrow[d, "\mathsf{G}(g_1)"] \\
\mathsf{F}(x_2) \arrow[d, "\mathsf{F}(f_2)"'] & \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"] \arrow[d, "F(f_2) \times \mathsf{G}(g_2)"] & \mathsf{G}(y_2) \arrow[d, "\mathsf{G}(g_2)"] \\
\mathsf{F}(x_3) & \mathsf{F}(x_3) \times_{\mathcal{D}} \mathsf{G}(y_3) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathsf{G}(y_3)
\end{tikzcd}$$

To show that $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2 \circ f_1, g_2 \circ g_1) = ((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2, g_2)) \circ ((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_1, g_1))$, it hence suffices to show that the following diagram commutes

$$\begin{tikzcd}
\mathsf{F}(x_1) \arrow[dd, "\mathsf{F}(f_2 \circ f_1)"'] & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) \arrow[l, "\pi_1"'] \arrow[r, "\pi_2"] \arrow[d, "F(f_1) \times \mathsf{G}(g_1)"] & \mathsf{G}(y_1) \arrow[dd, "\mathsf{G}(g_2 \circ g_1)"] \\
 & \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2) \arrow[d, "F(f_2) \times \mathsf{G}(g_2)"] & \\
\mathsf{F}(x_3) & \mathsf{F}(x_3) \times_{\mathcal{D}} \mathsf{G}(y_3) \arrow[l, "\pi_1"] \arrow[r, "\pi_2"'] & \mathsf{G}(y_3)
\end{tikzcd}$$

We only show that the left rectangle commutes, since proving the commutativity of the right rectangle is similar.

$$\begin{aligned}\pi_1 \circ \mathsf{F}(f_2) \times \mathsf{G}(g_2) \circ \mathsf{F}(f_1) \times \mathsf{G}(g_1) &= \mathsf{F}(f_2) \circ \pi_1 \circ \mathsf{F}(f_1) \times \mathsf{G}(g_1) \\ &= \mathsf{F}(f_2) \circ \mathsf{F}(f_1) \circ \pi_1 \\ &= \mathsf{F}(f_2 \circ f_1) \circ \pi_1\end{aligned}$$

Example 3.10. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{D}$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{D}$, and suppose that $\mathcal{D}$ has binary coproducts. We have a functor $\mathsf{F} +_{\mathcal{D}} \mathsf{G} : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{D}$, whose definition is dual to Example 3.9. We leave it to the reader to verify that we indeed have this functor.

3.2 Natural Transformations

Definition 3.11. Let $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $G : \mathcal{C}_1 \to \mathcal{C}_2$ be functors. A **natural transformation** $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ consists of a morphism $\tau(x) : \mathsf{F}(x) \to \mathsf{G}(x)$ such that for all $f : x \to y$ the following diagram commutes.

$$\begin{tikzcd} \mathsf{F}(x) \arrow[r, "\tau(x)"] \arrow[d, "\mathsf{F}(f)"'] & \mathsf{G}(x) \arrow[d, "\mathsf{G}(f)"] \\ \mathsf{F}(y) \arrow[r, "\tau(y)"'] & \mathsf{G}(y) \end{tikzcd}$$

This means that $\mathsf{G}(y) \circ \tau(x) = \tau(y) \circ \mathsf{F}(f)$.

Example 3.12. Let $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ be a functor. We define the **identity natural transformation** $\mathsf{id}_{\mathsf{F}} : \mathsf{F} \Rightarrow \mathsf{F}$ as follows. For each $x : \mathcal{C}_1$, we define $\mathsf{id}_{\mathsf{F}}(x) : \mathsf{F}(x) \to \mathsf{F}(x)$ to be the map $\mathsf{id}_{\mathsf{F}(x)}$. Note that we indeed have a natural transformation, because the following diagram commutes.

$$\begin{tikzcd} \mathsf{F}(x) \arrow[r, "\mathsf{id}_{\mathsf{F}(x)}"] \arrow[d, "\mathsf{F}(f)"'] & \mathsf{F}(x) \arrow[d, "\mathsf{F}(f)"] \\ \mathsf{F}(y) \arrow[r, "\mathsf{id}_{\mathsf{F}(y)}"'] & \mathsf{F}(y) \end{tikzcd}$$

This is because

$$\mathsf{F}(f) \circ \mathsf{id}_{\mathsf{F}(x)} = \mathsf{F}(f) = \mathsf{id}_{\mathsf{F}(y)} \circ \mathsf{F}(f)$$

Example 3.13. Suppose that we have three functors $\mathsf{F}, \mathsf{G}, \mathsf{H} : \mathcal{C}_1 \to \mathcal{C}_2$, and that we have natural transformations $\tau_1 : \mathsf{F} \Rightarrow \mathsf{G}$ and $\tau_2 : \mathsf{G} \Rightarrow \mathsf{H}$. We define their **composition** $\tau_2 \circ \tau_1 : \mathsf{F} \Rightarrow \mathsf{H}$ as follows. For each $x : \mathcal{C}_1$, we define $(\tau_2 \circ \tau_1)(x) : \mathsf{F}(x) \to \mathsf{H}(x)$ to be $\tau_2(x) \circ \tau_1(x)$. We indeed have a natural transformation, because the following diagram commutes.

$$\begin{tikzcd}
\mathsf{F}(x) \arrow[r, "\tau_1(x)"] \arrow[d, "\mathsf{F}(f)"'] & \mathsf{G}(x) \arrow[r, "\tau_2(x)"] \arrow[d, "\mathsf{G}(y)"] & \mathsf{H}(x) \arrow[d, "\mathsf{H}(f)"] \\
\mathsf{F}(y) \arrow[r, "\tau_1(y)"'] & \mathsf{G}(y) \arrow[r, "\tau_2(y)"'] & \mathsf{H}(y)
\end{tikzcd}$$

The left- and right squares commute, because τ_1 and τ_2 are natural transformations. We can also present this proof using equational reasoning as follows.

$$\begin{aligned}
\mathsf{H}(f) \circ \tau_2(x) \circ \tau_1(x) &= \tau_2(y) \circ \mathsf{G}(f) \circ \tau_1(x) \\
&= \tau_2(y) \circ \tau_1(y) \circ \mathsf{F}(f)
\end{aligned}$$

Example 3.14. Suppose we have functors $\mathsf{F}_1, \mathsf{F}_2 : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{C}_3$, and that we have a natural transformation $\tau : \mathsf{F}_1 \Rightarrow \mathsf{F}_2$. We define the **left whiskering** $\mathsf{G} \triangleleft \tau : \mathsf{G} \circ \mathsf{F}_1 \Rightarrow \mathsf{G} \circ \mathsf{F}_2$ as follows. For $x : \mathcal{C}_1$, we define $(\mathsf{G} \triangleleft \tau)(x) : \mathsf{G}(\mathsf{F}_1(x)) \to \mathsf{G}(\mathsf{F}_2(x))$ to be $\mathsf{G}(\tau(x))$. To prove naturality, we need to show the following diagram commutes for every $f : x \to y$.

$$\begin{tikzcd}
\mathsf{G}(\mathsf{F}_1(x)) \arrow[r, "\mathsf{G}(\tau(x))"] \arrow[d, "\mathsf{G}(\mathsf{F}_1(f))"'] & \mathsf{G}(\mathsf{F}_2(x)) \arrow[d, "\mathsf{G}(\mathsf{F}_2(f))"] \\
\mathsf{G}(\mathsf{F}_1(y)) \arrow[r, "\mathsf{G}(\tau(y))"'] & \mathsf{G}(\mathsf{F}_2(y))
\end{tikzcd}$$

This is because

$$\begin{aligned}
\mathsf{G}(\mathsf{F}_2(f)) \circ \mathsf{G}(\tau(x)) &= \mathsf{G}(\mathsf{F}_2(f) \circ \tau(x)) \\
&= \mathsf{G}(\tau(x) \circ \mathsf{F}_1(f)) \\
&= \mathsf{G}(\tau(x)) \circ \mathsf{G}(\mathsf{F}_1(f))
\end{aligned}$$

Example 3.15. Suppose we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G}_1, \mathsf{G}_2 : \mathcal{C}_2 \to \mathcal{C}_3$, and that we have a natural transformation $\tau : \mathsf{G}_1 \Rightarrow \mathsf{G}_2$. We define the **right whiskering** $\tau \triangleright F : \mathsf{G}_1 \circ \mathsf{F} \Rightarrow \mathsf{G}_2 \circ \mathsf{F}$ as follows. For $x : \mathcal{C}_1$, we define $(\tau \triangleright F)(x) : \mathsf{G}_1(\mathsf{F}(x)) \to \mathsf{G}_2(\mathsf{F}(x))$ to be $\tau(\mathsf{F}(x))$. To prove naturality, we need to show the following diagram commutes for every $f : x \to y$.

$$\begin{tikzcd}
\mathsf{G}_1(\mathsf{F}(x)) \arrow[r, "\tau(\mathsf{F}(x))"] \arrow[d, "\mathsf{G}_1(\mathsf{F}(f))"'] & \mathsf{G}_2(\mathsf{F}(x)) \arrow[d, "\mathsf{G}_2(\mathsf{F}(f))"] \\
\mathsf{G}_1(\mathsf{F}(y)) \arrow[r, "\tau(\mathsf{F}(y))"'] & \mathsf{G}_2(\mathsf{F}(y))
\end{tikzcd}$$

This diagram commutes, because τ is natural: it is the naturality square for $\mathsf{F}(f)$.

Example 3.16. Suppose we have functors $\mathsf{F}_1, \mathsf{F}_2 : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G}_1, \mathsf{G}_2 : \mathcal{C}_2 \to \mathcal{C}_3$, and that we have a natural transformations $\tau_1 : \mathsf{F}_1 \Rightarrow \mathsf{F}_2$ and $\tau_2 : \mathsf{G}_1 \Rightarrow \mathsf{G}_2$. Then we have a **horizontal composition** $\tau_1 * \tau_2 : \mathsf{G}_1 \circ \mathsf{F}_1 \Rightarrow \mathsf{G}_2 \circ \mathsf{F}_2$. We leave it to the reader to define this natural transformation.

Example 3.17. Recall the functor Const_y from Example 3.4. Let $f : y_1 \to y_2$ be a morphisms. We define the **constant natural transformation** $\mathsf{Const}_f : \mathsf{Const}_{y_1} \to \mathsf{Const}_{y_2}$ as follows. Given $x \in \mathcal{C}_1$, we define $\mathsf{Const}_f(x) : y_1 \to y_2$ to be f. To prove that this is indeed a natural transformation, we need to show that the following diagram commutes.

$$\begin{tikzcd}
y_1 \arrow[r, "f"] \arrow[d, "\mathsf{id}"'] & y_2 \arrow[d, "\mathsf{id}"] \\
y_1 \arrow[r, "f"'] & y_2
\end{tikzcd}$$

This is so, because

$$\mathsf{id} \circ f = f = f \circ \mathsf{id}$$

Example 3.18. Recall the functor Maybe from 3.5. We define a natural transformation $\eta_{\mathsf{Maybe}} : \mathsf{id}_{\mathsf{Set}} \Rightarrow \mathsf{Maybe}$. For each set X, we define $\eta_{\mathsf{Maybe}}(X) : X \Rightarrow X + \mathbb{1}$ to be the function that maps x to $(0, x)$. To prove naturality, we have to prove the following square commutes for each function $f : X \to Y$.

$$\begin{tikzcd}
X \arrow[r, "\eta_{\mathsf{Maybe}}(X)"] \arrow[d, "f"'] & X + \mathbb{1} \arrow[d, "f + \mathsf{id}_{\mathbb{1}}"] \\
Y \arrow[r, "\eta_{\mathsf{Maybe}}(Y)"'] & Y + \mathbb{1}
\end{tikzcd}$$

We take an arbitrary $x \in X$ and we note that

$$\begin{aligned}
((f + \mathsf{id}_{\mathbb{1}}) \circ \eta_{\mathsf{Maybe}}(X))(x) &= (f + \mathsf{id}_{\mathbb{1}})(\eta_{\mathsf{Maybe}}(X)(x)) \\
&= (f + \mathsf{id}_{\mathbb{1}})(0, x) \\
&= (0, f(x)) \\
&= \eta_{\mathsf{Maybe}}(Y)(f(x)) \\
&= (\eta_{\mathsf{Maybe}}(Y) \circ f)(x)
\end{aligned}$$

Example 3.19. We define a natural transformation $\mu_{\mathsf{Maybe}} : \mathsf{Maybe} \circ \mathsf{Maybe} \Rightarrow \mathsf{Maybe}$. For each set X, we need to define a morphism $\mu_{\mathsf{Maybe}}(X) : (X + \mathbb{1}) + \mathbb{1} \to X + \mathbb{1}$. If we have $z \in (X + \mathbb{1}) + \mathbb{1}$, then there are three cases:

$$z = (0, (0, x)), \qquad z = (0, (1, *)), \qquad z = (1, *)$$

We define $\mu_{\mathsf{Maybe}}(X) : X + \mathbb{1}$ as follows

$$\mu_{\mathsf{Maybe}}(X)(z) = \begin{cases} (0, x) & \text{if } z = (0, (0, x)) \text{ for some } x \in X \\ (0, x) & \text{otherwise} \end{cases}$$

To prove naturality, we need to show that the following diagram commutes for each $f : X \to Y$.

$$\begin{tikzcd}
(X + \mathbb{1}) + \mathbb{1} \arrow[r, "\mu_{\mathsf{Maybe}}(X)"] \arrow[d, "(f+\mathsf{id}_{\mathbb{1}})+\mathsf{id}_{unit}"'] & X + \mathbb{1} \arrow[d, "f+\mathsf{id}_{\mathbb{1}}"] \\
(Y + \mathbb{1}) + \mathbb{1} \arrow[r, "\mu_{\mathsf{Maybe}}(Y)"'] & Y + \mathbb{1}
\end{tikzcd}$$

Let $z \in (X + \mathbb{1}) + \mathbb{1}$ be any element. There are three cases. If $z = (0,(0,x))$, then we have

$$\begin{aligned}
((f + \mathsf{id}_{\mathbb{1}}) \circ \mu_{\mathsf{Maybe}}(X))(0,(0,x)) &= (f + \mathsf{id}_{\mathbb{1}})(\mu_{\mathsf{Maybe}}(X)(0,(0,x))) \\
&= (f + \mathsf{id}_{\mathbb{1}})(0,x) \\
&= (0, f(x)) \\
&= \mu_{\mathsf{Maybe}}(Y)(0,(0,f(x))) \\
&= \mu_{\mathsf{Maybe}}(Y)(((f + \mathsf{id}_{\mathbb{1}}) + \mathsf{id}_{unit})(0,(0,x))) \\
&= (\mu_{\mathsf{Maybe}}(Y) \circ ((f + \mathsf{id}_{\mathbb{1}}) + \mathsf{id}_{unit}))(0,(0,x))
\end{aligned}$$

The cases for $z = (0,(1,*)$ and $z = (1,*)$ are similar.

Digression. Examples 3.18 and 3.19 give the unit and multiplication of the maybe monad. The unit represents pure composition, whereas multiplication allows us to compose partial computation. We look at this example further in Section 6.

Example 3.20. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{D}$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{D}$, and suppose that $\mathcal{D}$ has binary products. Recall the functor $\mathsf{F} \times_D \mathsf{G}$ from Example 3.9. We define the **first projection** $\pi_1 : \mathsf{F} \times_{\mathcal{D}} \mathsf{G} \Rightarrow \mathsf{F}$ as follows. Given $x \in \mathcal{C}_1$ and $y \in \mathcal{C}_2$, we define $\pi_1(x,y) : \mathsf{F}(x) \times \mathsf{G}(y) \to \mathsf{F}(x)$. To prove naturality, we assume that we have morphisms $f : x_1 \to x_2$ and $g : y_1 \to y_2$ and we need to prove the following square commutes.

$$\begin{tikzcd}
\mathsf{F}(x_1) \times \mathsf{G}(y_1) \arrow[r, "\pi_1"] \arrow[d, "\mathsf{F}(f)\times\mathsf{G}(g)"'] & \mathsf{F}(x_1) \arrow[d, "\mathsf{F}(f)"] \\
\mathsf{F}(x_2) \times \mathsf{G}(y_2) \arrow[r, "\pi_1"'] & \mathsf{F}(x_2)
\end{tikzcd}$$

This diagram commutes because of how $\mathsf{F}(f) \times \mathsf{G}(g)$ is defined (Example 2.19).

We leave it as an exercise to define the **second projection** $\pi_2 : \mathsf{F} \times_{\mathcal{D}} \mathsf{G} \Rightarrow \mathsf{G}$
.

3.3 Categories of Functors

Definition 3.21. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be categories. We define the **functor category** $[\mathcal{C}_1, \mathcal{C}_2]$ as follows.

- Objects of $[\mathcal{C}_1, \mathcal{C}_2]$ are functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$.
- Morphisms from $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ to $\mathsf{G} : \mathcal{C}_1 \to \mathcal{C}_2$ are natural transformations $\tau : \mathsf{F} \Rightarrow \mathsf{G}$.

The identity morphism is the identity natural transformation (Example 3.12) and composition is the composition of natural transformations (Example 3.13).

Proposition 3.22. *We indeed have a category* $[\mathcal{C}_1, \mathcal{C}_2]$.

Proof. We need to prove associativity and unitality. If we have a natural transformation $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ where $\mathsf{F}, \mathsf{G} : \mathcal{C}_1 \to \mathcal{C}_2$, we need to show that $\tau circ \mathsf{id}_{\mathsf{F}} = \tau$. To do so, it suffices to show that for each $x : \mathcal{C}_1$ we have $(\tau circ \mathsf{id}_{\mathsf{F}})(x) = \tau(x)$. Note the following

$$\begin{aligned} (\tau circ \mathsf{id}_{\mathsf{F}})(x) &= \tau(x) \circ \mathsf{id}_{\mathsf{F}(x)} \\ &= \tau(x) \end{aligned}$$

We prove $\mathsf{id}_{\mathsf{G}} \circ \tau = \tau$ in the same way.

$$\begin{aligned} (\mathsf{id}_{\mathsf{G}} \circ \tau)(x) &= \mathsf{id}_{\mathsf{G}(x)} \circ \tau(x) \\ &= \tau(x) \end{aligned}$$

If we have natural transformations $\tau_1 : \mathsf{F} \Rightarrow \mathsf{G}$ and $\tau_2 : \mathsf{G} \Rightarrow \mathsf{H}$, then we show that $\tau_3 \circ (\tau_2 \circ \tau_1) = (\tau_3 \circ \tau_2) \circ \tau_1$. We take $x : \mathcal{C}_1$ and note that

$$\begin{aligned} (\tau_3 \circ (\tau_2 \circ \tau_1))(x) &= \tau_3(x) \circ (\tau_2 \circ \tau_1)(x) \\ &= \tau_3(x) \circ (\tau_2(x) \circ \tau_1(x)) \\ &= (\tau_3(x) \circ \tau_2(x)) \circ \tau_1(x) \\ &= (\tau_3 \circ \tau_2)(x) \circ \tau_1(x) \\ &= ((\tau_3 \circ \tau_2) \circ \tau_1)(x) \end{aligned}$$

$\square$

Proposition 3.23. *A natural transformation* $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ *is an isomorphism in* $[\mathcal{C}_1, \mathcal{C}_2]$ *if and only if* $\tau(x)$ *is an isomorphism in* $\mathcal{C}_1$ *for each* $x : \mathcal{C}_1$.

Proof. Suppose that $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ is an isomorphism. This means that we have a natural transformation $\theta : \mathsf{G} \Rightarrow \mathsf{F}$ such that $\tau \circ \theta = \mathsf{id}_{\mathsf{G}}$ and $\theta \circ \tau = \mathsf{id}_{\mathsf{F}}$. For each $x \in \mathcal{C}_1$, we thus have

- a morphism $\theta(x) : \mathsf{G}(x) \to \mathsf{F}(x)$
- we have $\tau(x) \circ \theta(x) = \mathsf{id}_{\mathsf{G}(x)}$ since $\tau \circ \theta = \mathsf{id}_{\mathsf{G}}$
- we have $\theta(x) \circ \tau(x) = \mathsf{id}_{\mathsf{F}(x)}$ since $\theta \circ \tau = \mathsf{id}_{\mathsf{F}}$

Hence, $\tau(x)$ is an isomorphism with inverse $\theta(x)$.

Now suppose that each $\tau(x) : \mathsf{F}(x) \to \mathsf{G}(x)$ is an isomorphism, which means that we have a morphism $\tau(x)^{-1} : \mathsf{G}(x) \to \mathsf{F}(x)$ such that

$$\tau(x) \circ \tau(x)^{-1} = \mathsf{id}_{\mathsf{G}(x)}, \qquad \tau(x)^{-1} \circ \tau(x) = \mathsf{id}_{\mathsf{F}(x)}$$

We construct a natural transformation $\theta : \mathsf{G} \Rightarrow \mathsf{F}$ as follows. For each $x \in \mathcal{C}_1$, we define $\theta(x) = \tau(x)^{-1}$. To show that θ is natural, we need to show the following diagram commutes for each $f : x \to y$.

$$\begin{tikzcd} \mathsf{G}(x) \arrow[r, "\tau(x)^{-1}"] \arrow[d, "\mathsf{G}(f)"'] & \mathsf{F}(x) \arrow[d, "\mathsf{F}(f)"] \\ \mathsf{G}(y) \arrow[r, "\tau(y)^{-1}"'] & \mathsf{F}(y) \end{tikzcd}$$

This is so, because

$$\begin{aligned} \mathsf{F}(f) \circ \tau(x)^{-1} &= \mathsf{id} \circ \mathsf{F}(f) \circ \tau(x)^{-1} \\ &= \tau(y)^{-1} \circ \tau(y) \circ \mathsf{F}(f) \circ \tau(x)^{-1} \\ &= \tau(y)^{-1} \circ \mathsf{G}(f) \circ \tau(x) \circ \tau(x)^{-1} \\ &= \tau(y)^{-1} \circ \mathsf{G}(f) \circ \mathsf{id} \\ &= \tau(y)^{-1} \circ \mathsf{G}(f) \end{aligned}$$

Note that we have $\tau \circ \theta = \mathsf{id}$ and $\theta \circ \tau = \mathsf{id}$. For each $x : \mathcal{C}_1$, we have

$$(\tau \circ \theta)(x) = \tau(x) \circ \tau(x)^{-1} = \mathsf{id}$$

$$(\theta \circ \tau)(x) = \tau(x)^{-1} \circ \tau(x) = \mathsf{id}$$

Hence, τ is an isomorphism in $[\mathcal{C}_1, \mathcal{C}_2]$. $\square$

Definition 3.24. We define the category Cat as follows.

- Objects of Cat are categories
- Morphisms from $\mathcal{C}_1$ to $\mathcal{C}_2$ are functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$

We define the identity morphism in Example 3.2 and composition in Example 3.3. We invite the reader to prove that neutrality and associativity laws.

Digression. There are two remarks to make about Definition 3.24. First, in the definition of category we require that there is a class of objects and a class of objects. To define this category, we need restrict ourselves to **small** categories: categories which have a set of objects and a set of morphisms. This is necessary to guarantee that we have a class of such categories.

Second, there is no mention of natural transformations in Definition 3.24 despite the fact that natural transformations are an essential part of category theory. This is the reason why one often looks at the **2-category of categories**. While a category has objects and morphisms (for instance, sets and functions), a 2-category has objects, morphisms, and 2-cells (for instance, categories, functors, and natural transformations). Studying categories using 2-categories gives one extra power, and it motivates the study of higher categories. One motivation for using higher categories is that we can express a wider range of universal properties. Universal properties in categories refer to objects and morphisms, whereas universal properties in 2-categories can refer to 2-cells as well.

3.4 Exercises

Exercise 3.1. Finish Example 3.7 by proving the functor laws for $\mathcal{L}_A$.

Exercise 3.2. Finish Example 3.8 by proving the functor laws.

Exercise 3.3. Finish Example 3.10 by defining the functor $\mathsf{F} +_{\mathcal{D}} \mathsf{G}$ and proving that it is indeed a functor.

Exercise 3.4. Complete Example 3.20 by constructing the second projection $\pi_2 : \mathsf{F} \times_{\mathcal{D}} \mathsf{G} \Rightarrow \mathsf{G}$.

Exercise 3.5. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$. Construct the functor $\mathsf{F}^{\mathsf{op}} : \mathcal{C}_1^{\mathsf{op}} \to \mathcal{C}_2^{\mathsf{op}}$ and prove that it is indeed a functor.

Exercise 3.6. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$, and that we have predicates P_1 on the objects of $\mathcal{C}_1$ and P_2 on the objects of $\mathcal{C}_2$. Construct a functor $\mathsf{Full}(\mathsf{F}) : \mathsf{Full}(\mathcal{C}_1, \mathsf{P}_1, \to)\mathsf{Full}(\mathcal{C}_2, \mathsf{P}_2,)$ whenever we have $\mathsf{P}_2(\mathsf{F}(x))$ if we have $\mathsf{P}_1(x)$.

Exercise 3.7. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$. Construct a functor $\mathsf{F}/x : \mathcal{C}_1/x \to \mathcal{C}_2/\mathsf{F}(x)$ for each $x \in \mathcal{C}_1$.

Exercise 3.8. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$. Construct a functor $\mathsf{F}^{\to} : \mathcal{C}_1^{\to} \to \mathcal{C}_2^{\to}$.

Exercise 3.9. Show that Cat has an initial object.

Exercise 3.10. Show that Cat has a terminal object.

Exercise 3.11. Show that Cat has binary products.

Exercise 3.12. Show that Cat has binary coproducts.

4 Algebras and Coalgebras

4.1 Algebras for a Functor

Definition 4.1. Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. An **algebra for** F consists of an object $x : \mathcal{C}$ together with a morphism $c : \mathsf{F}(x) \to x$. We call x the **carrier** of this algebra, and we say that c is an **F-algebra structure for** x.

Definition 4.2. Let (x_1, c_1) and (x_2, c_2) be algebras for F. An **algebra morphism from** (x_1, c_1) **to** (x_2, c_2) consists of a morphism $f : x_1 \to x_2$ such that the following diagram commutes.

$$\begin{tikzcd}
\mathsf{F}(x_1) \arrow[r, "\mathsf{F}(f)"] \arrow[d, "c_1"'] & \mathsf{F}(x_2) \arrow[d, "c_2"] \\
x_1 \arrow[r, "f"'] & x_2
\end{tikzcd}$$

We define the **category** Alg_F **of algebras for** F as follows.

- objects of Alg_F are algebras for F;
- morphisms from (x_1, c_1) to (x_2, c_2) are algebra morphisms.

Proposition 4.3. Alg_F *is indeed a category.*

Proof. We define the identity morphism on (x, c) as id_x.

$$\begin{tikzcd}
\mathsf{F}(x) \arrow[r, "\mathsf{F}(\mathsf{id}_x)"] \arrow[d, "c"'] & \mathsf{F}(x) \arrow[d, "c"] \\
x \arrow[r, "\mathsf{id}_x"'] & x
\end{tikzcd}$$

This diagram commutes because

$$\mathsf{id}_x \circ c = c = c \circ \mathsf{id}_{\mathsf{F}(x)} = c \circ \mathsf{F}(\mathsf{id}_x)$$

To define composition, we assume that we have an algebra morphism f from (x_1, c_1) to (x_2, c_2) and an algebra morphism g from (x_2, c_2) to (x_3, c_3). We display them below

$$\begin{tikzcd}
\mathsf{F}(x_1) \arrow[r, "\mathsf{F}(f)"] \arrow[d, "c_1"'] & \mathsf{F}(x_2) \arrow[r, "\mathsf{F}(g)"] \arrow[d, "c_2"] & \mathsf{F}(x_3) \arrow[d, "c_3"] \\
x_1 \arrow[r, "f"'] & x_2 \arrow[r, "g"'] & x_2
\end{tikzcd}$$

We define their composition to be $g \circ f$. Note that $g \circ f$ is indeed an algebra morphism, because

$$\begin{aligned} c_3 \circ \mathsf{F}(g \circ f) &= c_3 \circ \mathsf{F}(g) \circ \mathsf{F}(f) && \text{because } \mathsf{F} \text{ is a functor} \\ &= g \circ c_2 \circ \mathsf{F}(f) && \text{because } g \text{ is an algebra morphism} \\ &= g \circ f \circ c_1 && \text{because } f \text{ is an algebra morphism} \end{aligned}$$

Note that the category laws for Alg_F follow directly from the corresponding laws in $\mathcal{C}$. □

Example 4.4. We have a functor $\mathsf{U}_F : \mathsf{Alg}_\mathsf{F} \to \mathcal{C}$, called the **underlying functor**. It is defined as follows

- $\mathsf{U}_F(x, c) = x$ where (x, c) is an algebra for F
- $\mathsf{U}_F(f) = x$ where f is an algebra morphism from (x_1, c_1) to (x_2, c_2).

One can directly prove that U_F preserves the identity and composition.

To get familiarity with algebras, we look at some examples. We start by looking at algebras for the functor $\mathsf{Maybe} : \mathsf{Set} \to \mathsf{Set}$. Recall that $\mathsf{Maybe}(X)$ is defined to be $X + \mathbb{1}$.

Proposition 4.5. *Algebras for* Maybe *are the same as sets* X *together with an element* $z : X$ *and a function* $s : X \to X$.

Proof. We can prove this proposition using the universal property of the coproduct (Definition 2.11). Morphisms $f : X + Y \to Z$ are the same as pairs of morphisms $g_1 : X \to Z$ and $g_2 : Y \to Z$. Given $f : X + Y \to Z$, we define $g_1 = f \circ \iota_1$ and $g_2 = f \circ \iota_2$. Given $g_1 : X \to Z$ and $g_2 : Y \to Z$, we define $f = [g_1, g_2]$.

We also prove this proposition more concretely. We show that for every set X we have an isomorphism between the set of functions $c : X + \mathbb{1} \to X$ and the collection of elements $z : X$ and $s : X \to X$.

First, we define a map f that maps pairs of elements $z : X$ and functions $s : X \to X$ to functions $c : X + \mathbb{1} \to X$. We define a function $f(z, s) : X + \mathbb{1} \to X$ as follows

$$f(z, s)(w) = \begin{cases} s(x) & \text{if } w = (0, x) \text{ for some } x \in X \\ z & \text{if } w = (1, *) \end{cases}$$

Next we define a map g that maps functions $c : X + \mathbb{1} \to X$ to elements $z : X$ and functions $s : X \to X$. We write $g_1(c)$ for $c(1, *) : X$, and $g_2(c)$ for the function that maps x to $c(0, x) : X$. The function g maps every c to the pair $(g_1(c), g_2(c))$.

Note that f and g are inverses to each other. Suppose that we have $z : X$ and $s : X \to X$. Then $f(z, s)$ gives us a function then $g(f(z, s))$ is the pair $(g_1(f(z, s)), g_2(f(z, s)))$. Note

$$g_1(f(z, s)) = f(z, s)(1, *) = z$$

$$g_2(f(z,s)))(x) = f(z,s)(0,x) = s(x)$$

Next we assume that we have $c : X + \mathbb{1} \to X$, and any element $w \in X + \mathbb{1}$. There are two cases. If $w = (0, x)$, then we have

$$f(g(c))(0,x) = g_2(c)(x) = c(0,x)$$

$$f(g(c))(1,x) = g_1(c) = c(1,x)$$

Hence, f and g indeed form an isomorphism. $\square$

Example 4.6. We define an algebra for Maybe as follows.

- The carrier is $\mathbb{1}$
- The morphism $!_{\mathsf{Maybe}(\mathbb{1})} : \mathsf{Maybe}(\mathbb{1}) \to \mathbb{1}$.

We denote this algebra by $\mathbb{1}_{\mathsf{Maybe}}$.

Proposition 4.7. *The algebra $\mathbb{1}_{\mathsf{Maybe}}$ is terminal.*

Proof. Let (x, c) be any algebra for Maybe. Our goal is to show that there is a unique morphism $f : x \to \mathbb{1}$ making the following diagram commute.

$$\begin{tikzcd}
\mathsf{Maybe}(x) \arrow[r, "\mathsf{Maybe}(f)"] \arrow[d, "c"'] & \mathsf{Maybe}(\mathbb{1}) \arrow[d, "!_{\mathsf{Maybe}(\mathbb{1})}"] \\
x \arrow[r, "f"'] & \mathbb{1}
\end{tikzcd}$$

We first note that since $\mathbb{1}$ is the terminal object in Set (Example 2.3), all morphisms $f, g : x \to \mathbb{1}$ are equal. Hence, it suffices to show that we have a morphism $f : x \to \mathbb{1}$. We define f to be $!_x$. To show that $f = !_x$ is a morphism of algebras, we need to show that the following diagram commutes.

$$\begin{tikzcd}
\mathsf{Maybe}(x) \arrow[r, "\mathsf{Maybe}(!_x)"] \arrow[d, "c"'] & \mathsf{Maybe}(\mathbb{1}) \arrow[d, "!_{\mathsf{Maybe}(\mathbb{1})}"] \\
x \arrow[r, "!_x"'] & \mathbb{1}
\end{tikzcd}$$

Concretely, we need to show that the morphism $!_{\mathsf{Maybe}(\mathbb{1})} \circ \mathsf{Maybe}(!_x)$ and $!_x \circ c$ are equal. Since both are morphisms from $\mathsf{Maybe}(x)$ to $\mathbb{1}$ and since $\mathbb{1}$ are terminal, they must be equal. $\square$

Example 4.8. Let $k \in \mathbb{N}$ be any natural number with $k > 0$, and define $\mathbb{N}_k$ to be the set $\{i \in \mathbb{N} | i < k\}$. We construct an algebra for Maybe whose carrier is $\mathbb{N}_k$. To do so, we use Proposition 4.5. We define the map $s : \mathbb{N}_k \to \mathbb{N}_k$ as follows

$$s(i) = \begin{cases} i+1 & \text{if } i+1 < k \\ 0 & \text{otherwise} \end{cases}$$

We define $z \in \mathbb{N}_k$ to be 0. We denote this algebra by $(\mathbb{N}_k, 0, s)$.

Example 4.9. We contruct two Maybe-algebra structures for $\mathbb{Z}$. Note that we have a map $\mathsf{S}_{\mathbb{Z}} : \mathbb{Z} \to \mathbb{Z}$, which maps x to $x+1$, and a map $\mathsf{S}_{\mathbb{Z}} : \mathbb{Z} \to \mathbb{Z}$, which maps x to $x-1$. From these maps we get two Maybe-algebra structures, namely $(\mathbb{Z}, 0, \mathsf{S}_{\mathbb{Z}})$ and $(\mathbb{Z}, 0, \mathsf{P}_{\mathbb{Z}})$. Note that there are many other possibilities as well, for instance $(\mathbb{Z}, 37, \mathsf{S}_{\mathbb{Z}})$.

Example 4.10. We have the following Maybe-algebra: $(\mathbb{N}, \mathsf{S}, 0)$. Here $\mathsf{S}(n) = n+1$.

Proposition 4.11. $(\mathbb{N}, \mathsf{S}, 0)$ *is initial.*

Proof. Suppose that we have any algebra (X, s, z). Our goal is to show that we have a unique morphism $f : \mathbb{N} \to X$ that makes the following diagram commute.

$$\begin{tikzcd}
\mathbb{N} + \mathbb{1} \arrow[r, "f + \mathsf{id}_{\mathbb{1}}"] \arrow[d, "{[\mathsf{S}, 0]}"'] & X + \mathbb{1} \arrow[d, "{[s, z]}"] \\
\mathbb{N} \arrow[r, "f"'] & X
\end{tikzcd}$$

We define f using the following recursive definition.

$$f(n) = \begin{cases} z(*) & \text{if } n = 0 \\ s(f(k)) & \text{if } n = k+1 \end{cases}$$

Note that this f makes the diagram above commute. This is because

$$\begin{aligned}
[s, z]((f + \mathsf{id}_{\mathbb{1}})(0, n)) &= [s, z](0, f(n)) \\
&= s(f(n)) \\
&= f(\mathsf{S}(n)) \\
&= f([\mathsf{S}, 0](0, n))
\end{aligned}$$

$$\begin{aligned}
[s, z]((f + \mathsf{id}_{\mathbb{1}})(1, *)) &= [s, z](1, *) \\
&= z(*) \\
&= f(0) \\
&= f([\mathsf{S}, 0](1, *))
\end{aligned}$$

Now let $g : \mathbb{N} \to X$ be any other algebra morphism. We show by induction that for all $n \in \mathbb{N}$ we have $g(n) = f(n)$. For the base case, we take $n = 0$, and we note

$$\begin{aligned}
f(0) &= z(*) \\
&= [s, z](1, *) \\
&= [s, z]((g + \mathsf{id}_{\mathbb{1}})(1, *)) \\
&= ([s, z] \circ (g + \mathsf{id}_{\mathbb{1}}))(1, *) \\
&= (g \circ [\mathsf{S}, 0])(1, *) \\
&= g([\mathsf{S}, 0](1, *)) \\
&= g(0)
\end{aligned}$$

Here we use that g is an algebra morphism meaning that $g + \mathsf{id}_{\mathbb{1}} = g \circ [\mathsf{S}, 0]$.

For the inductive case, we assume that $g(n) = f(n)$, and we need to show that $g(n+1) = f(n+1)$. This follows from the following

$$\begin{aligned} f(n+1) &= s(f(n)) \\ &= s(g(n)) \\ &= [s,z](0, g(n)) \\ &= [s,z]((g + \mathsf{id}_{\mathbb{1}})(0,n)) \\ &= ([s,z] \circ (g + \mathsf{id}_{\mathbb{1}}))(0,n) \\ &= g([\mathsf{S},0](0,n)) \\ &= g(n+1) \end{aligned}$$

Hencee $g = f$, and thus $(\mathbb{N}, \mathsf{S}, 0)$ is initial. □

Digression. The proof of Proposition 4.11 is interesting, and explains the relevance of initial algebras. The fact that $(\mathbb{N}, \mathsf{S}, 0)$ captures the fact that we can define certain recursive functions using $\mathbb{N}$. Specifically, initiality for Maybe captures the following recursive definition for functions $f : \mathbb{N} \to X$.

$$f(n) = \begin{cases} z & \text{if } n = 0 \\ s(f(k)) & \text{if } n = k+1 \end{cases}$$

Here $z \in X$ is some fixed element and $s : X \to X$ is a map. This recursive definition is the **iteration principle** for the natural numbers, which intuitively says that we can define functions $f : \mathbb{N} \to X$ by iterating some given function $s : X \to X$ on some input $z \in X$.

From the perspective of programming, initial algebras allows us to describe inductive data types: we describe the constructor using a functor, and initiality gives us a recursion principle.

Next we look at algebra for the functor $\mathcal{L}_A$. Given a set A, recall that $\mathcal{L}_A(X)$ is defined to be $X \times A + \mathbb{1}$. We can characterise algebras for $\mathcal{L}_A$ as follows.

Proposition 4.12. *Algebras for $\mathcal{L}_A$ are the same as sets X together with an element $n : X$ and a function $c : X \times A \to X$.*

Proof. Proven in the same way as Proposition 4.5. □

Example 4.13. We construct a $\mathcal{L}_A$-algebra. Its carrier is $\mathbb{N}$, we take $n : \mathbb{N}$ to be 0, and we define $c : \mathbb{N} \times A \to \mathbb{N}$ as follows

$$c(n, a) = \mathsf{S}(x)$$

Example 4.14. We consider the set $\mathsf{List}(A)$ of lists with elements from A.

$$\mathsf{List}(A) = \{(a_1, \ldots, a_n) \mid a_i \in A\}$$

Note that n could be 0 meaning that we have an element $() \in \mathsf{List}(A)$, and we denote this element by Nil. We construct an $\mathcal{L}_A$-algebra structure on $\mathsf{List}(A)$. The element $n : \mathsf{List}(A)$ is defined to be Nil, and the operation $c : \mathsf{List}(A) \times A \to \mathsf{List}(A)$ is defined as the following function

$$\mathsf{Cons}((a_1, \ldots, a_n), b) = (b, a_1, \ldots, a_n)$$

We denote this algebra by $(\mathsf{List}(A), \mathsf{Nil}, \mathsf{Cons})$.

Proposition 4.15. $(\mathsf{List}(A), \mathsf{Nil}, \mathsf{Cons})$ *is initial.*

Proof. Let (X, n, c) be any $\mathcal{L}_A$-algebra. We need to show that there is a unique morphism f that makes the following diagram commutes.

$$\begin{tikzcd}
\mathsf{List}(A) \times A + \mathbb{1} \arrow[r, "(f \times \mathsf{id}_A) + \mathsf{id}_{\mathbb{1}}"] \arrow[d, "{[\mathsf{Cons}, \mathsf{Nil}]}"'] & X \times A + \mathbb{1} \arrow[d, "{[c, n]}"] \\
\mathsf{List}(A) \arrow[r, "f"'] & X
\end{tikzcd}$$

To do so, we first note that for every $l \in \mathsf{List}(A)$ we either have that $l = \mathsf{Nil}$ or there is $l' \in \mathsf{List}(A)$ and $a \in A$ such that $l = \mathsf{Cons}(l', a)$. We define f using induction over lists.

$$f(l) = \begin{cases} n & \text{if } l = \mathsf{Nil} \\ c(f(l'), a) & \text{if } l = \mathsf{Cons}(l', a) \end{cases}$$

We leave it as an exercise to the reader to prove that f is an $\mathcal{L}_A$-algebra morphism.

Next we show that f is the unique algebra morphism with this property. Let $g : \mathsf{List}(A) \to X$ be any other $\mathcal{L}_A$-algebra morphism. We need to show that for each $l \in \mathsf{List}(A)$, we have $g(l) = f(l)$, and to do so, we use induction over lists. The base is $l = \mathsf{Nil}$, and then we have

$$f(\mathsf{Nil}) = n = g(\mathsf{Nil})$$

We have $g(\mathsf{Nil}) = n$, because g is an $\mathcal{L}_A$-algebra morphism.

For the induction step, we assume that we have a list $l' \in \mathsf{List}(A)$ with $g(l') = f(l')$ and $a \in A$. We need to show

$$\begin{aligned} f(\mathsf{Cons}(l', a)) &= c(f(l'), a) \\ &= c(g(l'), a) \\ &= g(\mathsf{Cons}(l', a)) \end{aligned}$$

Since g is an algebra morphism, we indeed have $c(g(l'), a) = g(\mathsf{Cons}(l', a))$. $\square$

Example 4.16. We can combine Proposition 4.15 and Example 4.13 to obtain a morphism $\mathsf{length} : \mathsf{List}(A) \to \mathbb{N}$. By inspecting the proof of Proposition 4.15, we see that length is defined as follows.

$$\mathsf{length}(l) = \begin{cases} 0 & \text{if } l = \mathsf{Nil} \\ \mathsf{length}(l') + 1 & \text{if } l = \mathsf{Cons}(l', a) \end{cases}$$

Hence, the function length calculates the length of a list.

4.2 Coalgebras for a Functor

Definition 4.17. Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. A **coalgebra for** F consists of an object $x : \mathcal{C}$ together with a morphism $d : x \to \mathsf{F}(x)$. We call x the **carrier** of this coalgebra, and we say that d is an **F-coalgebra structure for** x.

Let (x_1, d_1) and (x_2, d_2) be coalgebras for F. A **coalgebra morphism from** (x_1, d_1) **to** (x_2, d_2) consists of a morphism $f : x_1 \to x_2$ such that the following diagram commutes.

$$\begin{tikzcd} x_1 \arrow[r, "f"] \arrow[d, "d_1"'] & x_2 \arrow[d, "d_2"] \\ \mathsf{F}(x_1) \arrow[r, "\mathsf{F}(f)"'] & \mathsf{F}(x_2) \end{tikzcd}$$

We define the **category** $\mathsf{CoAlg}_\mathsf{F}$ **of coalgebras for** F as follows.

- objects of $\mathsf{CoAlg}_\mathsf{F}$ are coalgebras for F;
- morphisms from (x_1, d_1) to (x_2, d_2) are coalgebra morphisms.

Example 4.18. We have a functor $\mathsf{U}_F : \mathsf{CoAlg}_\mathsf{F} \to \mathcal{C}$, called the **underlying functor**. It is defined as follows

- $\mathsf{U}_F(x, c) = x$ where (x, d) is an coalgebra for F
- $\mathsf{U}_F(f) = x$ where f is an coalgebra morphism from (x_1, c_1) to (x_2, c_2).

One can directly prove that U_F preserves the identity and composition.

Example 4.19. Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. This functor F induces a functor $\mathsf{F}^{\mathsf{op}} : \mathcal{C}^{\mathsf{op}} \to \mathcal{C}^{\mathsf{op}}$ (Exercise 3.5). A coalgebra for F^{op} consists of an object $x : \mathcal{C}$ together with a morphism $d : x \to_{\mathcal{C}^{\mathsf{op}}} \mathsf{F}(x)$, and d gives a morphism $d : \mathsf{F}(x) \to_{\mathcal{C}} x$. Hence, coalgebras for F^{op} are the same as algebras for F.

As a concrete example, we look at algebras for the functor $(-) \times A : \mathsf{Set} \to \mathsf{Set}$ (Example 3.6), which was defined to be $((-) \times A)(X) = X \times A$. Note that coalgebras for $(-) \times A$ consists of a set X together with two maps $t : X \to X$ and $h : X \to A$.

Example 4.20. We show that we have a coalgebras for $(-) \times A$ whose carrier is A. We need to construct a $t : A \to A$ and $h : A \to A$, and for both we take the identity id_A. We denote this coalgebra by $(A, \mathsf{id}_A, \mathsf{id}_A)$.

Example 4.21. We define the set $\mathsf{Stream}(A)$ to be the set of all functions $f : \mathbb{N} \to A$. The function $\mathsf{Head} : \mathsf{Stream}(A) \to A$ maps $f : \mathbb{N} \to A$ to $f(0) : A$. Given $f : \mathbb{N} \to A$, we define $\mathsf{Tail}(f) : \mathbb{N} \to A$ to be $\mathsf{Tail}(f)(n) = f(n+1)$. This coalgebra is denoted by $(\mathsf{Stream}(A), \mathsf{Head}, \mathsf{Tail})$.

Proposition 4.22. $(\mathsf{Stream}(A), \mathsf{Head}, \mathsf{Tail})$ *is terminal.*

Proof. Let (X, h, t) be any coalgebra for $(-) \times A$. We need to show there is a unique function $f : X \to \mathsf{Stream}(A)$ that makes the following diagram commutes.

$$\begin{tikzcd} X \arrow[r, "f"] \arrow[d, "\langle t,h\rangle"'] & \mathsf{Stream}(A) \arrow[d, "\langle \mathsf{Tail},\mathsf{Head}\rangle"] \\ X \times A \arrow[r, "f \times \mathsf{id}_A"'] & \mathsf{Stream}(A) \times A \end{tikzcd}$$

We define f as follows

$$f(x)(n) = h(t^n(x))$$

Note

$$\mathsf{Head}(f(x)) = f(x)(0) = h(x)$$

$$\mathsf{Tail}(f(x))(n) = f(x)(n+1) = h(t^{n+1}(x)) = f(t(x))(n)$$

The uniqueness of f is left as an exercise. $\square$

Example 4.23. We can combine Proposition 4.22 and 4.20 to obtain a function $\mathsf{const}_{\mathsf{Stream}} : A \to \mathsf{Stream}(A)$. By unfolding the description given in the proof of Proposition 4.22, we see that $\mathsf{const}_{\mathsf{Stream}}$ maps every a to as follows

$$\mathsf{const}_{\mathsf{Stream}}(a)(n) = \mathsf{id}(\mathsf{id}^n(a)) = a$$

Hence, this function maps every a to the constant function.

Digression. We discussed that initial algebras allow us to view inductive data types categorically. The constructors are specified using the functor F and initiality gives us a recursion principle. With terminal coalgebras one can study coinductive data types. Coinductive types are specified using destructors rather than constructors. For instance, the type $\mathsf{Stream}(A)$ of streams is specified by two destructors. We have $\mathsf{Head} : \mathsf{Stream}(A) \to A$ that picks the first element and $\mathsf{Tail} : \mathsf{Stream}(A) \to \mathsf{Stream}(A)$ that removes the first element from the stream. Terminality allows us to define streams using corecursion.

Digression. One nice aspect of studying Cat as a 2-category is that we can characterise certain categorical constructions via 2-categorical universal properties. In particular, we can characterise $\mathsf{Alg}_(F)$ and $\mathsf{CoAlg}_(F)$ via a 2-categorical universal property.

$$\begin{tikzcd} \mathsf{Alg}_(\mathsf{F}) \arrow[r, "\mathsf{U_F}"] & \mathcal{C} \arrow[r, shift left, "\mathsf{F}"] \arrow[r, shift right, "\mathsf{Id}_{\mathcal{C}}"'] & \mathcal{C} \end{tikzcd}$$

We can construct a natural transformation $\tau : \mathsf{F} \circ \mathsf{U_F} \Rightarrow \mathsf{Id}_{\mathcal{C}} \circ \mathsf{U_F}$. Given an algebra (X, c), we define $\tau(X, c) : F(X) \to X$ to be c. The universal property of $\mathsf{Alg_F}$ allows us to make functors $\mathsf{H} : \mathcal{D} \to \mathsf{Alg_F})$ in the folloiwing way.

$$
\begin{tikzcd}
\mathsf{Alg_F} \arrow[r, "\mathsf{U_F}"] & \mathcal{C} \arrow[r, shift left, "\mathsf{F}"] \arrow[r, shift right, "\mathsf{Id}_{\mathcal{C}}"'] & \mathcal{C} \\
\mathcal{D} \arrow[u, "\exists \mathsf{H}"] \arrow[ur, "\mathsf{G}"'] & &
\end{tikzcd}
$$

For coalgebras, we have a similar universal property.

$$
\begin{tikzcd}
\mathsf{CoAlg}_F \arrow[r, "\mathsf{U_F}"] & \mathcal{C} \arrow[r, shift left, "\mathsf{Id}_{\mathcal{C}}"] \arrow[r, shift right, "\mathsf{F}"'] & \mathcal{C} \\
\mathcal{D} \arrow[u, dashed, "\exists \mathsf{H}"] \arrow[ur, "\mathsf{G}"'] & &
\end{tikzcd}
$$

4.3 Exercises

Exercise 4.1. Prove Proposition 4.12.

Exercise 4.2. Show that CoAlg_F is a category by constructing the identity morphisms and composition, and by showing the laws.

Exercise 4.3. Finish the proof of Proposition 4.22 by proving that f is the unique coalgebra

Exercise 4.4. Let $\mathcal{C}$ be a category with a terminal object, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that $\mathsf{Alg_F}$ has a terminal object.

Exercise 4.5. Let $\mathcal{C}$ be a category with binary products, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that $\mathsf{Alg_F}$ has binary products.

Exercise 4.6. Let $\mathcal{C}$ be a category with an initial object, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that $\mathsf{CoAlg_F}$ has a terminal object.

Exercise 4.7. Let $\mathcal{C}$ be a category with binary coproducts, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that $\mathsf{CoAlg_F}$ has binary coproducts.

Exercise 4.8. Let $a : A$. Use Proposition 4.15 to construct a map $\mathsf{const_{List}} : \mathbb{N} \to \mathsf{List}(A)$ that maps every n to the list $(a, \ldots, a)$ with length n. Prove that $\mathsf{length} \circ \mathsf{const_{List}} = \mathsf{id}_{\mathbb{N}}$.

Exercise 4.9. Let $f : A \to B$ be a function. Use Proposition 4.15 to construct a map $\mathsf{List}(f) : \mathsf{List}(A) \to \mathsf{List}(B)$. Prove that $\mathsf{List}(\mathsf{id}_A) = \mathsf{id}_{\mathsf{List}(A)}$ and that $\mathsf{List}(g \circ f) = \mathsf{List}(g) \circ \mathsf{List}(f)$.

Exercise 4.10. Using Proposition 4.22 to define a function $o : \mathsf{Stream}(A) \to \mathsf{Stream}(A)$ that maps $f : \mathbb{N} \to A$ to the function $g(n) = f(2 \cdot n + 1)$.

Exercise 4.11. Let $f : A \to B$ be a function. Use Proposition 4.22 to construct a map $\mathsf{Stream}(f) : \mathsf{Stream}(A) \to \mathsf{Stream}(B)$. Prove that $\mathsf{Stream}(\mathsf{id}_A) = \mathsf{id}_{\mathsf{Stream}(A)}$ and that $\mathsf{Stream}(g \circ f) = \mathsf{Stream}(g) \circ \mathsf{Stream}(f)$.

5 Adjunctions

6 Monads

7 Presheaves and the Yoneda Lemma

Digression. λ-calculus: terms as a presheaf

8 Cartesian Closed Categories

9 Cartesian Closed Categories and the λ-Calculus

10 Hyperdoctrines

11 Quantifiers as Adjoints

12 Triposes and Higher-Order Logic